

Programming with Python

2. Einleitung

Thomas Weise (汤卫思)
tweise@hfuu.edu.cn

School of Artificial Intelligence and Big Data
Hefei University
Hefei, Anhui, China

人工智能与大数据学院
合肥大学
中国安徽省合肥市

Programming with Python



Dies ist ein Kurs über das Programmieren mit der Programmiersprache Python an der Universität Hefei (合肥大学).

Die Webseite mit dem Lehrmaterial dieses Kurses ist <https://thomasweise.github.io/programmingWithPython> (siehe auch den QR-Code unten rechts). Dort können Sie das Kursbuch (in Englisch) und diese Slides finden. Das Repository mit den Beispielprogrammen in Python finden Sie unter <https://github.com/thomasWeise/programmingWithPythonCode>.



Outline



1. Einleitung
2. Programmieren vs. Softwareentwicklung
3. Warum Python?
4. Literatur
5. Zusammenfassung





Einleitung



Einleitung



- Dieser Kurs lehrt das Programmieren mit der Programmiersprache Python.

Einleitung



- Dieser Kurs lehrt das Programmieren mit der Programmiersprache Python.
- Was bedeutet *Programmieren*?

Einleitung



- Dieser Kurs lehrt das Programmieren mit der Programmiersprache Python.
- Was bedeutet *Programmieren*?
- Programmieren bedeutet, dass wir Aufgaben an den Computer delegieren.

Einleitung



- Dieser Kurs lehrt das Programmieren mit der Programmiersprache Python.
- Was bedeutet *Programmieren*?
- Programmieren bedeutet, dass wir Aufgaben an den Computer delegieren.
- Wir haben eine Aufgabe zu erledigen, irgendeine Sache.

Einleitung



- Dieser Kurs lehrt das Programmieren mit der Programmiersprache Python.
- Was bedeutet *Programmieren*?
- Programmieren bedeutet, dass wir Aufgaben an den Computer delegieren.
- Wir haben eine Aufgabe zu erledigen, irgendeine Sache.
- Vielleicht ist sie zu kompliziert und dauert zulange.

Einleitung



- Dieser Kurs lehrt das Programmieren mit der Programmiersprache Python.
- Was bedeutet *Programmieren*?
- Programmieren bedeutet, dass wir Aufgaben an den Computer delegieren.
- Wir haben eine Aufgabe zu erledigen, irgendeine Sache.
- Vielleicht ist sie zu kompliziert und dauert zulange.
- Vielleicht ist es etwas, das wir sehr oft machen müssen.

Einleitung



- Dieser Kurs lehrt das Programmieren mit der Programmiersprache Python.
- Was bedeutet *Programmieren*?
- Programmieren bedeutet, dass wir Aufgaben an den Computer delegieren.
- Wir haben eine Aufgabe zu erledigen, irgendeine Sache.
- Vielleicht ist sie zu kompliziert und dauert zulange.
- Vielleicht ist es etwas, das wir sehr oft machen müssen.
- Vielleicht ist es etwas, das wir physisch nicht selbst machen können.

Einleitung



- Dieser Kurs lehrt das Programmieren mit der Programmiersprache Python.
- Was bedeutet *Programmieren*?
- Programmieren bedeutet, dass wir Aufgaben an den Computer delegieren.
- Wir haben eine Aufgabe zu erledigen, irgendeine Sache.
- Vielleicht ist sie zu kompliziert und dauert zulange.
- Vielleicht ist es etwas, das wir sehr oft machen müssen.
- Vielleicht ist es etwas, das wir physisch nicht selbst machen können.
- Vielleicht sind wir einfach faul.

Einleitung



- Dieser Kurs lehrt das Programmieren mit der Programmiersprache Python.
- Was bedeutet *Programmieren*?
- Programmieren bedeutet, dass wir Aufgaben an den Computer delegieren.
- Wir haben eine Aufgabe zu erledigen, irgendeine Sache.
- Vielleicht ist sie zu kompliziert und dauert zulange.
- Vielleicht ist es etwas, das wir sehr oft machen müssen.
- Vielleicht ist es etwas, das wir physisch nicht selbst machen können.
- Vielleicht sind wir einfach faul.
- Also wollen wir, dass der Computer es für uns macht.

Einleitung



- Wenn wir eine Aufgabe an eine andere Person delegieren, dann müssen wir die Aufgabe erklären.

Einleitung



- Wenn wir eine Aufgabe an eine andere Person delegieren, dann müssen wir die Aufgabe erklären.
- Wenn Sie der Chefkoch in einer Küche sind, dann müssen Sie dem Azubikoch erklären: „Erst musst Du die Kartoffeln waschen, dann schälen, und dann kannst Du sie kochen.“

Einleitung



- Wenn wir eine Aufgabe an eine andere Person delegieren, dann müssen wir die Aufgabe erklären.
- Wenn Sie der Chefkoch in einer Küche sind, dann müssen Sie dem Azubikoch erklären: „Erst musst Du die Kartoffeln waschen, dann schälen, und dann kannst Du sie kochen.“
- Wenn Sie zum Friseur gehen um sich die Haare schön machen zu lassen, dann sagen Sie zum Beispiel: „Wasch meine Haare, schneide sie oben auf 1cm, trimm die Seiten, und färbe sie grün.“

Einleitung



- Wenn wir eine Aufgabe an eine andere Person delegieren, dann müssen wir die Aufgabe erklären.
- Wenn Sie der Chefkoch in einer Küche sind, dann müssen Sie dem Azubikoch erklären: „Erst musst Du die Kartoffeln waschen, dann schälen, und dann kannst Du sie kochen.“
- Wenn Sie zum Friseur gehen um sich die Haare schön machen zu lassen, dann sagen Sie zum Beispiel: „Wasch meine Haare, schneide sie oben auf 1cm, trimm die Seiten, und färbe sie grün.“
- Wir geben der anderen Person eine klare und eindeutige Sequenz von Anweisungen in einer Sprache, die sie versteht.

Einleitung



- Wenn wir eine Aufgabe an eine andere Person delegieren, dann müssen wir die Aufgabe erklären.
- Wenn Sie der Chefkoch in einer Küche sind, dann müssen Sie dem Azubikoch erklären: „Erst musst Du die Kartoffeln waschen, dann schälen, und dann kannst Du sie kochen.“
- Wenn Sie zum Friseur gehen um sich die Haare schön machen zu lassen, dann sagen Sie zum Beispiel: „Wasch meine Haare, schneide sie oben auf 1cm, trimm die Seiten, und färbe sie grün.“
- Wir geben der anderen Person eine klare und eindeutige Sequenz von Anweisungen in einer Sprache, die sie versteht.
- In diesem Kurs lernen Sie, wie Sie das selbe mit einem Computer machen können.



Programmieren vs. Softwareentwicklung



Definition: Programm

Ein *Programm* ist eine eindeutige Sequenz von Berechnungsanweisungen für einen Computer um ein spezifisches Ziel zu erreichen.

Definition: Programm

Ein *Programm* ist eine eindeutige Sequenz von Berechnungsanweisungen für einen Computer um ein spezifisches Ziel zu erreichen.

Definition: Programmieren

Programmieren ist das Schreiben eines Programms⁸⁵.

Programmieren



- In der überwältigenden Mehrheit der Fälle erstellen wir ein Programm *nicht*, um es nur ein einziges Mal auszuführen.

Programmieren



- In der überwältigenden Mehrheit der Fälle erstellen wir ein Programm *nicht*, um es nur ein einziges Mal auszuführen.
- Wenn wir Aufgaben im realen Leben delegieren, ist das ja ganz ähnlich.

Programmieren



- In der überwältigenden Mehrheit der Fälle erstellen wir ein Programm *nicht*, um es nur ein einziges Mal auszuführen.
- Wenn wir Aufgaben im realen Leben delegieren, ist das ja ganz ähnlich.
- Als Chefkoch „geben“ Sie das „Programm“ *Kartoffeln kochen* in den Azubikoch ja auch nur einmal „ein.“

Programmieren



- In der überwältigenden Mehrheit der Fälle erstellen wir ein Programm *nicht*, um es nur ein einziges Mal auszuführen.
- Wenn wir Aufgaben im realen Leben delegieren, ist das ja ganz ähnlich.
- Als Chefkoch „geben“ Sie das „Programm“ *Kartoffeln kochen* in den Azubikoch ja auch nur einmal „ein.“
- Danach wollen Sie in der Lage sein, dieses „Programm“ auszuführen, in dem Sie sagen: „Koch doch bitte 2kg Kartoffeln.“

Programmieren



- In der überwältigenden Mehrheit der Fälle erstellen wir ein Programm *nicht*, um es nur ein einziges Mal auszuführen.
- Wenn wir Aufgaben im realen Leben delegieren, ist das ja ganz ähnlich.
- Als Chefkoch „geben“ Sie das „Programm“ *Kartoffeln kochen* in den Azubikoch ja auch nur einmal „ein.“
- Danach wollen Sie in der Lage sein, dieses „Programm“ auszuführen, in dem Sie sagen: „Koch doch bitte 2kg Kartoffeln.“
- Solche „Programme“ haben also sogar oft implizite Parameter, wie zum Beispiel das eben erwähnte Gewicht von 2kg.

Programmieren



- In der überwältigenden Mehrheit der Fälle erstellen wir ein Programm *nicht*, um es nur ein einziges Mal auszuführen.
- Wenn wir Aufgaben im realen Leben delegieren, ist das ja ganz ähnlich.
- Als Chefkoch „geben“ Sie das „Programm“ *Kartoffeln kochen* in den Azubikoch ja auch nur einmal „ein.“
- Danach wollen Sie in der Lage sein, dieses „Programm“ auszuführen, in dem Sie sagen: „Koch doch bitte 2kg Kartoffeln.“
- Solche „Programme“ haben also sogar oft implizite Parameter, wie zum Beispiel das eben erwähnte Gewicht von 2kg.
- Wenn Sie das nächste Mal zum Friseur gehen, wollen Sie vielleicht sagen können: „Das selbe wie immer, aber heute färbe sie blau.“

Programmieren



- In unseren täglichen Interaktionen erfolgt das Erstellen von wiederverwendbaren, parameterisierten Programmen sehr oft und sehr implizit.

Programmieren



- In unseren täglichen Interaktionen erfolgt das Erstellen von wiederverwendbaren, parameterisierten Programmen sehr oft und sehr implizit.
- Wir denken darüber selten explizit nach.

Programmieren



- In unseren täglichen Interaktionen erfolgt das Erstellen von wiederverwendbaren, parameterisierten Programmen sehr oft und sehr implizit.
- Wir denken darüber selten explizit nach.
- Wenn wir jedoch Computer programmieren, dann denken wir sehr wohl explizit darüber nach.

Programmieren



- In unseren täglichen Interaktionen erfolgt das Erstellen von wiederverwendbaren, parameterisierten Programmen sehr oft und sehr implizit.
- Wir denken darüber selten explizit nach.
- Wenn wir jedoch Computer programmieren, dann denken wir sehr wohl explizit darüber nach.
- Gleich von Anfang an.

Programmieren



- In unseren täglichen Interaktionen erfolgt das Erstellen von wiederverwendbaren, parameterisierten Programmen sehr oft und sehr implizit.
- Wir denken darüber selten explizit nach.
- Wenn wir jedoch Computer programmieren, dann denken wir sehr wohl explizit darüber nach.
- Gleich von Anfang an.
- Programmieren ist aber nur ein Teil von Softwareentwicklung.

Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.



Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.



Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.



Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.



Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach?



Softwareentwicklung



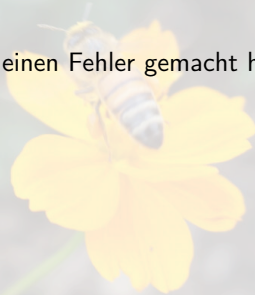
- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? **Nein.**



Softwareentwicklung



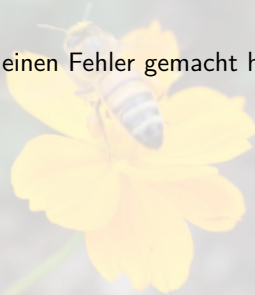
- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben.



Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler.



Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler. Je schwieriger die Aufgabe ist, die wir lösen wollen, je mehr Programmcode wir schreiben, desto wahrscheinlicher ist es, dass wir irgendwo irgendeinen Fehler machen.

Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler. Je schwieriger die Aufgabe ist, die wir lösen wollen, je mehr Programmcode wir schreiben, desto wahrscheinlicher ist es, dass wir irgendwo irgendeinen Fehler machen. **Also müssen Sie Ihre Programme testen.**

Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler. Je schwieriger die Aufgabe ist, die wir lösen wollen, je mehr Programmcode wir schreiben, desto wahrscheinlicher ist es, dass wir irgendwo irgendeinen Fehler machen. Also müssen Sie Ihre Programme testen.
 2. Was passiert wenn jemand anders Ihre Programme später verwenden will?

Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler. Je schwieriger die Aufgabe ist, die wir lösen wollen, je mehr Programmcode wir schreiben, desto wahrscheinlicher ist es, dass wir irgendwo irgendeinen Fehler machen. Also müssen Sie Ihre Programme testen.
 2. Was passiert wenn jemand anders Ihre Programme später verwenden will? **Sie müssen eine klare Dokumentation schreiben.**

Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler. Je schwieriger die Aufgabe ist, die wir lösen wollen, je mehr Programmcode wir schreiben, desto wahrscheinlicher ist es, dass wir irgendwo irgendeinen Fehler machen. Also müssen Sie Ihre Programme testen.
 2. Was passiert wenn jemand anders Ihre Programme später verwenden will? Sie müssen eine klare Dokumentation schreiben.
 3. Was, wenn Ihr Kode Funktionen zur Verfügung stellt, die andere verwenden können?

Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler. Je schwieriger die Aufgabe ist, die wir lösen wollen, je mehr Programmcode wir schreiben, desto wahrscheinlicher ist es, dass wir irgendwo irgendeinen Fehler machen. Also müssen Sie Ihre Programme testen.
 2. Was passiert wenn jemand anders Ihre Programme später verwenden will? Sie müssen eine klare Dokumentation schreiben.
 3. Was, wenn Ihr Code Funktionen zur Verfügung stellt, die andere verwenden können? **Die Ein- und Ausgabedatentypen müssen klar spezifiziert werden.**

Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler. Je schwieriger die Aufgabe ist, die wir lösen wollen, je mehr Programmcode wir schreiben, desto wahrscheinlicher ist es, dass wir irgendwo irgendeinen Fehler machen. Also müssen Sie Ihre Programme testen.
 2. Was passiert wenn jemand anders Ihre Programme später verwenden will? Sie müssen eine klare Dokumentation schreiben.
 3. Was, wenn Ihr Code Funktionen zur Verfügung stellt, die andere verwenden können? Die Ein- und Ausgabedatentypen müssen klar spezifiziert werden.
 4. Was, wenn jemand anders Ihren Code lesen und damit arbeiten soll?

Softwareentwicklung



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler. Je schwieriger die Aufgabe ist, die wir lösen wollen, je mehr Programmcode wir schreiben, desto wahrscheinlicher ist es, dass wir irgendwo irgendeinen Fehler machen. Also müssen Sie Ihre Programme testen.
 2. Was passiert wenn jemand anders Ihre Programme später verwenden will? Sie müssen eine klare Dokumentation schreiben.
 3. Was, wenn Ihr Code Funktionen zur Verfügung stellt, die andere verwenden können? Die Ein- und Ausgabedatentypen müssen klar spezifiziert werden.
 4. Was, wenn jemand anders Ihren Code lesen und damit arbeiten soll? **Ihr Code muss klar sein und konsistent einem ordentlichen Stil folgen¹⁰⁸.**



- Später, in Ihrem Job, wollen Sie ein Programm entwickeln, das eine bestimmte Aufgabe löst.
 1. Sie schreiben das Program.
 2. Dann haben Sie die Datei mit dem Programmcode.
 3. Das Problem ist gelöst.
- Ist das so einfach? Nein.
 1. Vielleicht fragen Sie sich, ob Sie einen Fehler gemacht haben. Leute machen Fehler. Je schwieriger die Aufgabe ist, die wir lösen wollen, je mehr Programmcode wir schreiben, desto wahrscheinlicher ist es, dass wir irgendwo irgendeinen Fehler machen. Also müssen Sie Ihre Programme testen.
 2. Was passiert wenn jemand anders Ihre Programme später verwenden will? Sie müssen eine klare Dokumentation schreiben.
 3. Was, wenn Ihr Code Funktionen zur Verfügung stellt, die andere verwenden können? Die Ein- und Ausgabedatentypen müssen klar spezifiziert werden.
 4. Was, wenn jemand anders Ihren Code lesen und damit arbeiten soll? Ihr Code muss klar sein und konsistent einem ordentlichen Stil folgen¹⁰⁸.
- Alle diese Dinge müssen beachtet werden!

Softwareentwicklung



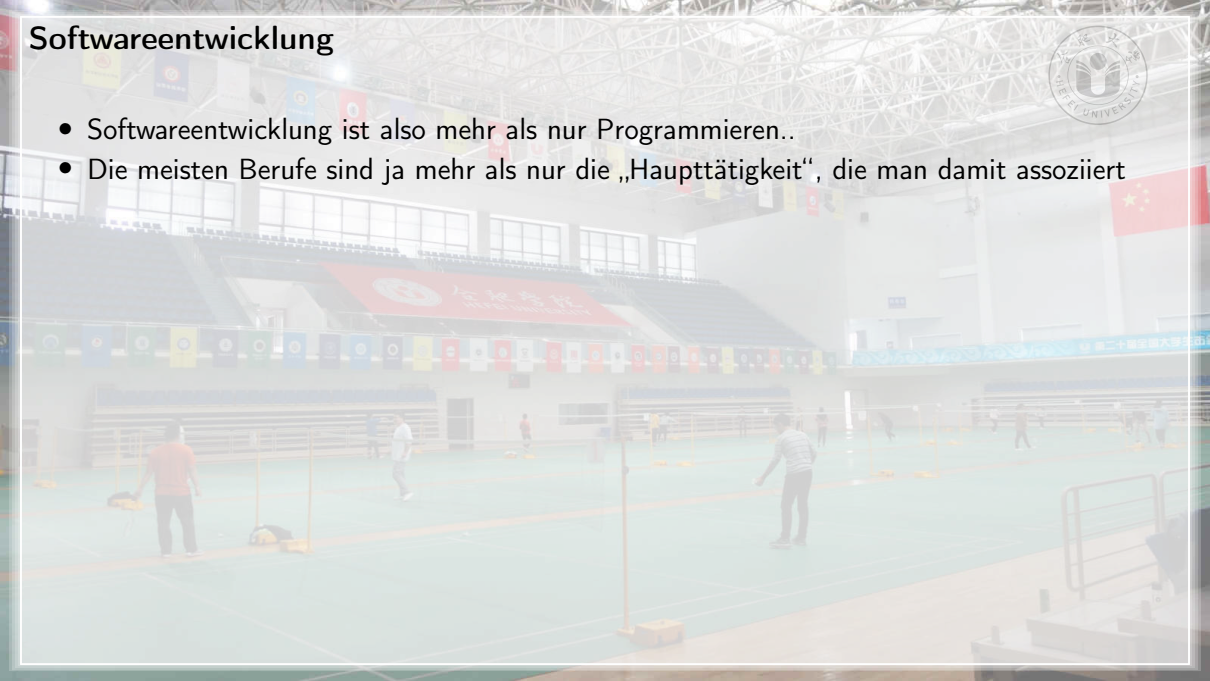
- Softwareentwicklung ist also mehr als nur Programmieren..



Softwareentwicklung



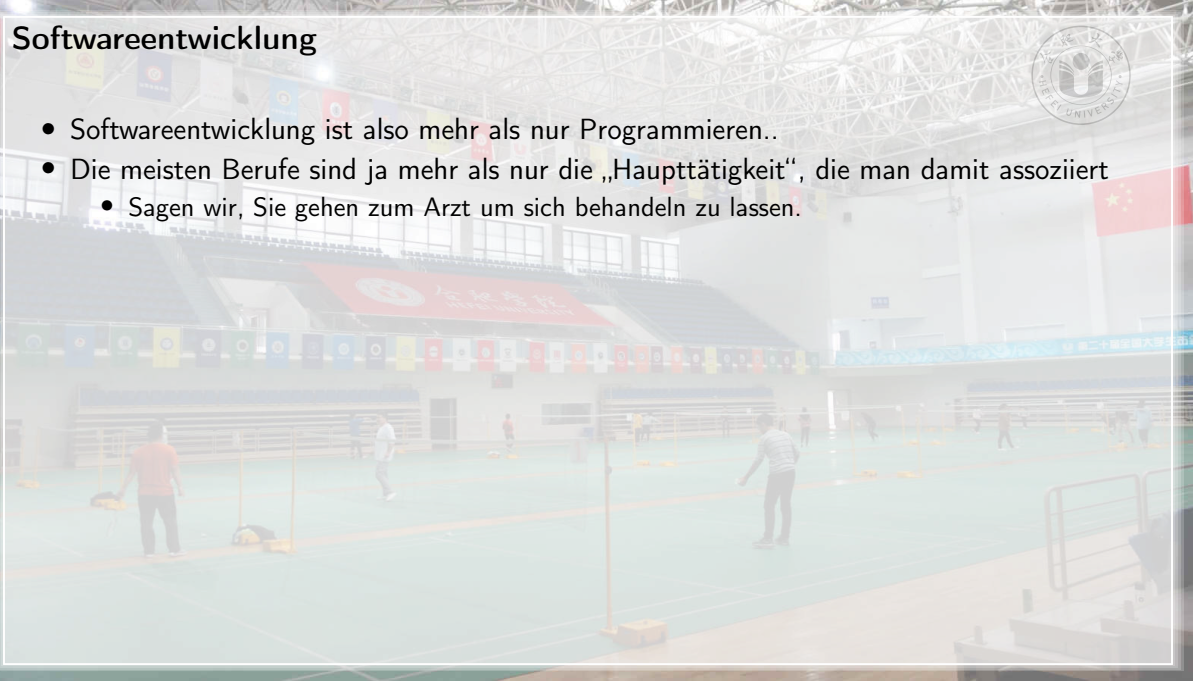
- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert



Softwareentwicklung



- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert
 - Sagen wir, Sie gehen zum Arzt um sich behandeln zu lassen.



Softwareentwicklung



- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert
 - Sagen wir, Sie gehen zum Arzt um sich behandeln zu lassen.
 - Dann *hoffen* Sie, dass dieser gut ausgebildet ist, die entsprechenden Operationen durchzuführen.

Softwareentwicklung



- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert
 - Sagen wir, Sie gehen zum Arzt um sich behandeln zu lassen.
 - Dann *hoffen* Sie, dass dieser gut ausgebildet ist, die entsprechenden Operationen durchzuführen.
 - Aber Sie *erwarten absolut*, dass er sich die Hände vor der Operation wäscht.

Softwareentwicklung



- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert
 - Sagen wir, Sie gehen zum Arzt um sich behandeln zu lassen.
 - Dann *hoffen* Sie, dass dieser gut ausgebildet ist, die entsprechenden Operationen durchzuführen.
 - Aber Sie *erwarten absolut*, dass er sich die Hände vor der Operation wäscht.
- Für Programmierer gilt das selbe!

Softwareentwicklung



- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert
 - Sagen wir, Sie gehen zum Arzt um sich behandeln zu lassen.
 - Dann *hoffen* Sie, dass dieser gut ausgebildet ist, die entsprechenden Operationen durchzuführen.
 - Aber Sie *erwarten absolut*, dass er sich die Hände vor der Operation wäscht.
- Für Programmierer gilt das selbe!
 - Sagen wir, Ihr Chef will, dass Sie ein Programm schreiben.

Softwareentwicklung



- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert
 - Sagen wir, Sie gehen zum Arzt um sich behandeln zu lassen.
 - Dann *hoffen* Sie, dass dieser gut ausgebildet ist, die entsprechenden Operationen durchzuführen.
 - Aber Sie *erwarten absolut*, dass er sich die Hände vor der Operation wäscht.
- Für Programmierer gilt das selbe!
 - Sagen wir, Ihr Chef will, dass Sie ein Programm schreiben.
 - Dann *hofft* er, dass Sie ein Programm schreiben, das gut funktioniert.

Softwareentwicklung



- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert
 - Sagen wir, Sie gehen zum Arzt um sich behandeln zu lassen.
 - Dann *hoffen* Sie, dass dieser gut ausgebildet ist, die entsprechenden Operationen durchzuführen.
 - Aber Sie *erwarten absolut*, dass er sich die Hände vor der Operation wäscht.
- Für Programmierer gilt das selbe!
 - Sagen wir, Ihr Chef will, dass Sie ein Programm schreiben.
 - Dann *hofft* er, dass Sie ein Programm schreiben, das gut funktioniert.
 - Aber er *erwartet*, dass der Code den Sie produzieren lesbar ist, das Sie ihn getestet haben, und dass sie ihn dokumentiert haben.

Softwareentwicklung



- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert
 - Sagen wir, Sie gehen zum Arzt um sich behandeln zu lassen.
 - Dann *hoffen* Sie, dass dieser gut ausgebildet ist, die entsprechenden Operationen durchzuführen.
 - Aber Sie *erwarten absolut*, dass er sich die Hände vor der Operation wäscht.
- Für Programmierer gilt das selbe!
 - Sagen wir, Ihr Chef will, dass Sie ein Programm schreiben.
 - Dann *hofft* er, dass Sie ein Programm schreiben, das gut funktioniert.
 - Aber er *erwartet*, dass der Code den Sie produzieren lesbar ist, das Sie ihn getestet haben, und dass sie ihn dokumentiert haben.
- Ich will nicht zu einem Arzt gehen, der sich nicht die Hände wäscht, bevor er mich operiert.

- Softwareentwicklung ist also mehr als nur Programmieren..
- Die meisten Berufe sind ja mehr als nur die „Haupttätigkeit“, die man damit assoziiert
 - Sagen wir, Sie gehen zum Arzt um sich behandeln zu lassen.
 - Dann *hoffen* Sie, dass dieser gut ausgebildet ist, die entsprechenden Operationen durchzuführen.
 - Aber Sie *erwarten absolut*, dass er sich die Hände vor der Operation wäscht.
- Für Programmierer gilt das selbe!
 - Sagen wir, Ihr Chef will, dass Sie ein Programm schreiben.
 - Dann *hofft* er, dass Sie ein Programm schreiben, das gut funktioniert.
 - Aber er *erwartet*, dass der Code den Sie produzieren lesbar ist, das Sie ihn getestet haben, und dass sie ihn dokumentiert haben.
- Ich will nicht zu einem Arzt gehen, der sich nicht die Hände wäscht, bevor er mich operiert.
- Ich will Ihnen nicht Programmieren beibringen, ohne den Fokus auf *sauberen* Code zu legen.

- Programmierer schreiben also nicht nur Kode, sie entwickeln Software.

- Programmierer schreiben also nicht nur Code, sie entwickeln Software.
- Ein Großteil der Programmierer verbringt nur etwa 50% ihrer Zeit mit Programmieren^{22,63}.

- Programmierer schreiben also nicht nur Code, sie entwickeln Software.
- Ein Großteil der Programmierer verbringt nur etwa 50% ihrer Zeit mit Programmieren^{22,63}.
- Andere Studien stellen sogar fest, dass weniger als 20% der Arbeitszeit mit Programmieren verbracht wird und vielleicht weitere 15% mit dem Korrigieren von Fehlern⁷⁰.

- Programmierer schreiben also nicht nur Code, sie entwickeln Software.
- Ein Großteil der Programmierer verbringt nur etwa 50% ihrer Zeit mit Programmieren^{22,63}.
- Andere Studien stellen sogar fest, dass weniger als 20% der Arbeitszeit mit Programmieren verbracht wird und vielleicht weitere 15% mit dem Korrigieren von Fehlern⁷⁰.
- Natürlich fokussieren wir uns in diesem Kurs auf das Programmieren.



- Programmierer schreiben also nicht nur Code, sie entwickeln Software.
- Ein Großteil der Programmierer verbringt nur etwa 50% ihrer Zeit mit Programmieren^{22,63}.
- Andere Studien stellen sogar fest, dass weniger als 20% der Arbeitszeit mit Programmieren verbracht wird und vielleicht weitere 15% mit dem Korrigieren von Fehlern⁷⁰.
- Natürlich fokussieren wir uns in diesem Kurs auf das Programmieren.
- Aber wir werden auch viele Dinge diskutieren, die darüber hinausgehen.



- Programmierer schreiben also nicht nur Code, sie entwickeln Software.
- Ein Großteil der Programmierer verbringt nur etwa 50% ihrer Zeit mit Programmieren^{22,63}.
- Andere Studien stellen sogar fest, dass weniger als 20% der Arbeitszeit mit Programmieren verbracht wird und vielleicht weitere 15% mit dem Korrigieren von Fehlern⁷⁰.
- Natürlich fokussieren wir uns in diesem Kurs auf das Programmieren.
- Aber wir werden auch viele Dinge diskutieren, die darüber hinausgehen. Dinge, die in Ihren Werkzeuggürtel gehören.



- Programmierer schreiben also nicht nur Code, sie entwickeln Software.
- Ein Großteil der Programmierer verbringt nur etwa 50% ihrer Zeit mit Programmieren^{22,63}.
- Andere Studien stellen sogar fest, dass weniger als 20% der Arbeitszeit mit Programmieren verbracht wird und vielleicht weitere 15% mit dem Korrigieren von Fehlern⁷⁰.
- Natürlich fokussieren wir uns in diesem Kurs auf das Programmieren.
- Aber wir werden auch viele Dinge diskutieren, die darüber hinausgehen. Dinge, die in Ihren Werkzeuggürtel gehören. Dinge, die Sie zu *guten* Programmierern machen.

- Programmierer schreiben also nicht nur Code, sie entwickeln Software.
- Ein Großteil der Programmierer verbringt nur etwa 50% ihrer Zeit mit Programmieren^{22,63}.
- Andere Studien stellen sogar fest, dass weniger als 20% der Arbeitszeit mit Programmieren verbracht wird und vielleicht weitere 15% mit dem Korrigieren von Fehlern⁷⁰.
- Natürlich fokussieren wir uns in diesem Kurs auf das Programmieren.
- Aber wir werden auch viele Dinge diskutieren, die darüber hinausgehen. Dinge, die in Ihren Werkzeuggürtel gehören. Dinge, die Sie zu *guten* Programmierern machen.
- Das Thema unseres Kurs ist das Entwickeln *guter* Software mit Python.



- Programmierer schreiben also nicht nur Code, sie entwickeln Software.
- Ein Großteil der Programmierer verbringt nur etwa 50% ihrer Zeit mit Programmieren^{22,63}.
- Andere Studien stellen sogar fest, dass weniger als 20% der Arbeitszeit mit Programmieren verbracht wird und vielleicht weitere 15% mit dem Korrigieren von Fehlern⁷⁰.
- Natürlich fokussieren wir uns in diesem Kurs auf das Programmieren.
- Aber wir werden auch viele Dinge diskutieren, die darüber hinausgehen. Dinge, die in Ihren Werkzeuggürtel gehören. Dinge, die Sie zu *guten* Programmierern machen.
- Das Thema unseres Kurs ist das Entwickeln *guter* Software **mit Python**.



Warum Python?



Warum Python?

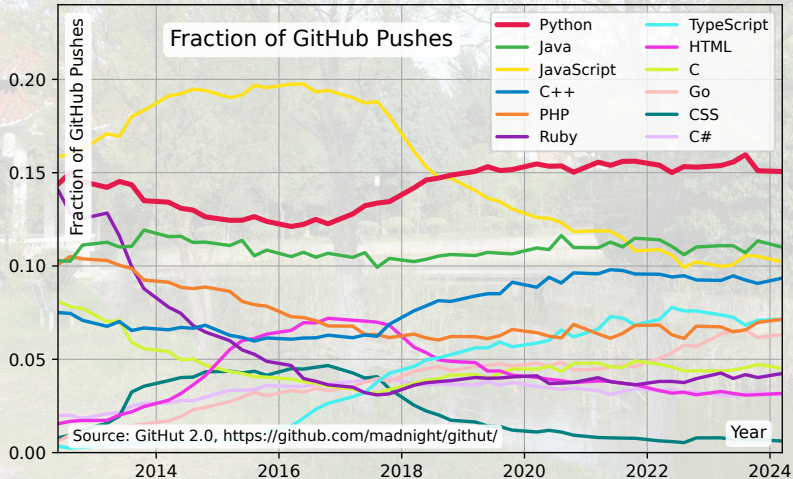
1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}.



Warum Python?



1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}.



Warum Python?



1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}. Nach der jährlichen Stack Overflow Umfrage 2024¹⁰⁰, war Python die zweitpopulärste Programmiersprache nach JavaScript and HTML/CSS.

Warum Python?



1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}. Nach der jährlichen Stack Overflow Umfrage 2024¹⁰⁰, war Python die zweitpopulärste Programmiersprache nach JavaScript and HTML/CSS. In GitHub's Octoverse Report vom Oktober 2024³², war Python die populärste Programmiersprache (vor JavaScript).

Warum Python?



1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}. Nach der jährlichen Stack Overflow Umfrage 2024¹⁰⁰, war Python die zweitpopulärste Programmiersprache nach JavaScript and HTML/CSS. In GitHub's Octoverse Report vom Oktober 2024³², war Python die populärste Programmiersprache (vor JavaScript).
2. Python wird intensiv auf dem Gebiet der AI⁹³, ML⁹⁶, und DS³⁸ genutzt, die alle Zukunftstechnologien sind.

Warum Python?



1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}. Nach der jährlichen Stack Overflow Umfrage 2024¹⁰⁰, war Python die zweitpopulärste Programmiersprache nach JavaScript and HTML/CSS. In GitHub's Octoverse Report vom Oktober 2024³², war Python die populärste Programmiersprache (vor JavaScript).
2. Python wird intensiv auf dem Gebiet der AI⁹³, ML⁹⁶, und DS³⁸ genutzt, die alle Zukunftstechnologien sind.
3. Es existieren sehr mächtige Bibliotheken sowohl für Forschung als auch für die Produktentwicklung, z. B. NumPy^{21,40,47}, Pandas^{5,57}, Scikit-learn^{82,88}, SciPy^{47,110}, TensorFlow^{1,52}, PyTorch^{81,88}, Matplotlib^{44,47,77}, SimPy¹²², und moptipy¹¹⁵, um nur ein paar zu nennen.

Warum Python?



1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}. Nach der jährlichen Stack Overflow Umfrage 2024¹⁰⁰, war Python die zweitpopulärste Programmiersprache nach JavaScript and HTML/CSS. In GitHub's Octoverse Report vom Oktober 2024³², war Python die populärste Programmiersprache (vor JavaScript).
2. Python wird intensiv auf dem Gebiet der AI⁹³, ML⁹⁶, und DS³⁸ genutzt, die alle Zukunftstechnologien sind.
3. Es existieren sehr mächtige Bibliotheken sowohl für Forschung als auch für die Produktentwicklung, z. B. NumPy^{21,40,47}, Pandas^{5,57}, Scikit-learn^{82,88}, SciPy^{47,110}, TensorFlow^{1,52}, PyTorch^{81,88}, Matplotlib^{44,47,77}, SimPy¹²², und moptipy¹¹⁵, um nur ein paar zu nennen.
4. Python ist sehr einfach zu erlernen^{37,107}.

Warum Python?



1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}. Nach der jährlichen Stack Overflow Umfrage 2024¹⁰⁰, war Python die zweitpopulärste Programmiersprache nach JavaScript and HTML/CSS. In GitHub's Octoverse Report vom Oktober 2024³², war Python die populärste Programmiersprache (vor JavaScript).
2. Python wird intensiv auf dem Gebiet der AI⁹³, ML⁹⁶, und DS³⁸ genutzt, die alle Zukunftstechnologien sind.
3. Es existieren sehr mächtige Bibliotheken sowohl für Forschung als auch für die Produktentwicklung, z. B. NumPy^{21,40,47}, Pandas^{5,57}, Scikit-learn^{82,88}, SciPy^{47,110}, TensorFlow^{1,52}, PyTorch^{81,88}, Matplotlib^{44,47,77}, SimPy¹²², und moptipy¹¹⁵, um nur ein paar zu nennen.
4. Python ist sehr einfach zu erlernen^{37,107}. Es hat eine einfache und saubere Syntax, die zu einer gut lesbaren Programmstruktur führt.

Warum Python?



1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}. Nach der jährlichen Stack Overflow Umfrage 2024¹⁰⁰, war Python die zweitpopulärste Programmiersprache nach JavaScript and HTML/CSS. In GitHub's Octoverse Report vom Oktober 2024³², war Python die populärste Programmiersprache (vor JavaScript).
2. Python wird intensiv auf dem Gebiet der AI⁹³, ML⁹⁶, und DS³⁸ genutzt, die alle Zukunftstechnologien sind.
3. Es existieren sehr mächtige Bibliotheken sowohl für Forschung als auch für die Produktentwicklung, z. B. NumPy^{21,40,47}, Pandas^{5,57}, Scikit-learn^{82,88}, SciPy^{47,110}, TensorFlow^{1,52}, PyTorch^{81,88}, Matplotlib^{44,47,77}, SimPy¹²², und moptipy¹¹⁵, um nur ein paar zu nennen.
4. Python ist sehr einfach zu erlernen^{37,107}. Es hat eine einfache und saubere Syntax, die zu einer gut lesbaren Programmstruktur führt. Python hat auch sehr mächtige Standarddatentypen, wie z. B. Listen, Tuples, und Dictionaries.

Warum Python?



1. Python ist eine sehr weitverbreitete Programmiersprache^{8,13}. Nach der jährlichen Stack Overflow Umfrage 2024¹⁰⁰, war Python die zweitpopulärste Programmiersprache nach JavaScript and HTML/CSS. In GitHub's Octoverse Report vom Oktober 2024³², war Python die populärste Programmiersprache (vor JavaScript).
2. Python wird intensiv auf dem Gebiet der AI⁹³, ML⁹⁶, und DS³⁸ genutzt, die alle Zukunftstechnologien sind.
3. Es existieren sehr mächtige Bibliotheken sowohl für Forschung als auch für die Produktentwicklung, z. B. NumPy^{21,40,47}, Pandas^{5,57}, Scikit-learn^{82,88}, SciPy^{47,110}, TensorFlow^{1,52}, PyTorch^{81,88}, Matplotlib^{44,47,77}, SimPy¹²², und moptipy¹¹⁵, um nur ein paar zu nennen.
4. Python ist sehr einfach zu erlernen^{37,107}. Es hat eine einfache und saubere Syntax, die zu einer gut lesbaren Programmstruktur führt. Python hat auch sehr mächtige Standarddatentypen, wie z. B. Listen, Tuples, und Dictionaries. Darum war Python auch die populärste Programmiersprache für Leute, die das Programmieren lernen wollen, nach der oben erwähnten Umfrage¹⁰⁰.

Python ist eine interpretierte Sprache

- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.



Python ist ein interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.

**Programmieren in einer kompilierten
Sprache wie C**

Programmieren

Python ist eine interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.

**Programmieren in einer kompilierten
Sprache wie C**



Python ist eine interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.

**Programmieren in einer kompilierten
Sprache wie C**

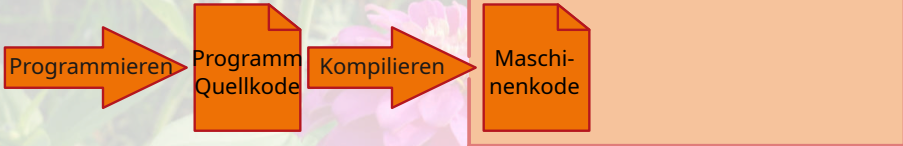


Python ist eine interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.

Programmieren in einer kompilierten Sprache wie C

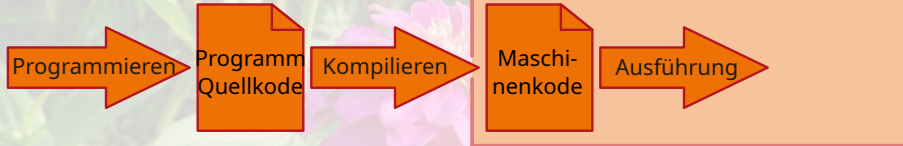


Python ist eine interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.

Programmieren in einer kompilierten Sprache wie C

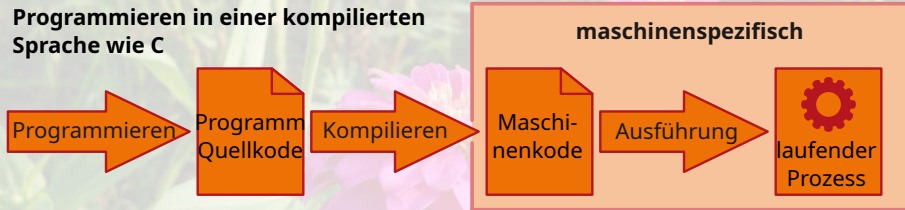


Python ist eine interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.

Programmieren in einer kompilierten Sprache wie C

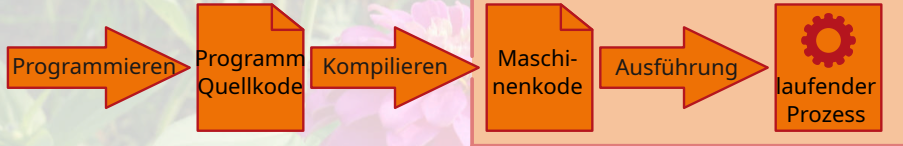


Python ist ein interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.
- Python ist interpretiert.

Programmieren in einer kompilierten Sprache wie C



Programmieren in Python

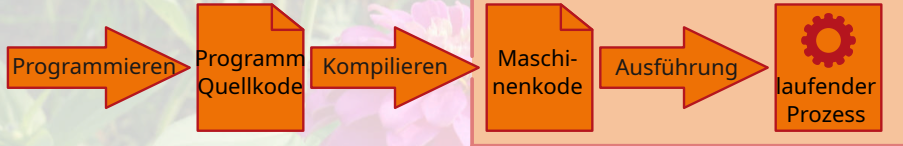


Python ist ein interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.
- Python ist interpretiert.

Programmieren in einer kompilierten Sprache wie C



Programmieren in Python

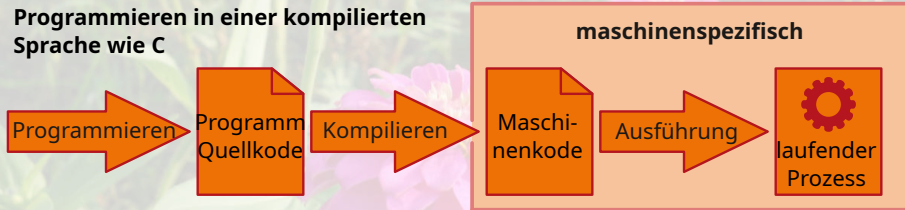


Python ist eine interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.
- Python ist interpretiert.

Programmieren in einer kompilierten Sprache wie C



Programmieren in Python

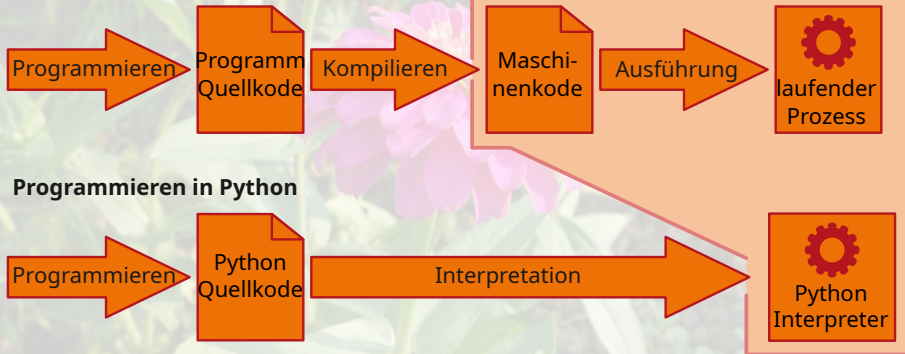


Python ist ein interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.
- Python ist interpretiert.

Programmieren in einer kompilierten Sprache wie C

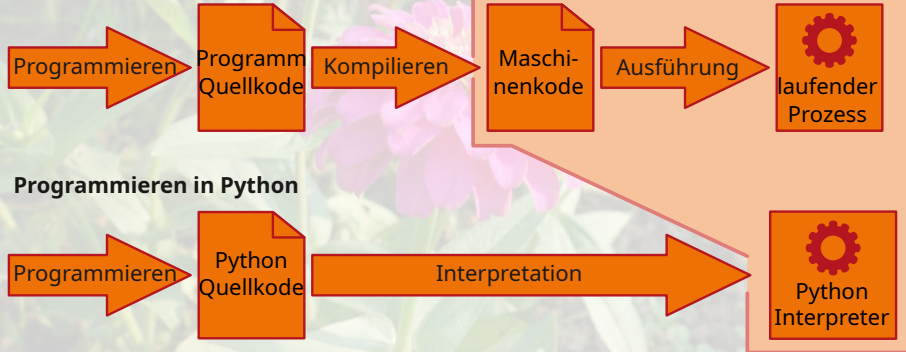


Python ist eine interpretierte Sprache



- Die meisten Programmiersprachen erfordern, dass Quellcode kompiliert wird.
- Python ist interpretiert.
- Daher gibt es weniger Schritte im Buildprozess.

Programmieren in einer kompilierten Sprache wie C





Literatur



Literatur

- Für diesen Kurs reicht unser Kursbuch¹¹³.



Literatur



- Für diesen Kurs reicht unser Kursbuch¹¹³.
- Weitere Bücher zu Python:
 - Bernd Klein. *Einführung in Python 3 – Für Ein- und Umsteiger*. 3., überarbeitete. Carl Hanser Verlag GmbH & Co. KG, 2018. ISBN: 978-3-446-45208-4. doi:10.3139/9783446453876,
 - Mark Lutz. *Learning Python*. 6. Aufl. O'Reilly Media, Inc., März 2025. ISBN: 978-1-0981-7130-8,
 - Kevin Wilson. *Python Made Easy*. Packt Publishing Ltd, Aug. 2024. ISBN: 978-1-83664-615-0,
 - Brett Slatkin. *Effective Python: 125 Specific Ways to Write Better Python*. 3. Aufl. Addison-Wesley Professional, Nov. 2024. ISBN: 978-0-13-817239-8,
 - John Hunt. *A Beginners Guide to Python 3 Programming*. 2. Aufl. Springer, 2023. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8,
 - Eric Matthes. *Python Crash Course*. 3. Aufl. No Starch Press, Jan. 2023. ISBN: 978-1-7185-0270-3,
 - Paul Barry. *Head First Python*. 3. Aufl. O'Reilly Media, Inc., Aug. 2023. ISBN: 978-1-4920-5129-9,
 - Mike James. *Programmer's Python: Everything is an Object – Something Completely Different*. 2. Aufl. I/O Press, 25. Juni 2022.

Literatur



- Für diesen Kurs reicht unser Kursbuch¹¹³.
- Weitere Bücher zu Softwaretests und kontinuierliche Integration:
 - Brian Okken. *Python Testing with pytest*. Pragmatic Bookshelf by The Pragmatic Programmers, L.L.C., Feb. 2022. ISBN: 978-1-68050-860-4,
 - Ashwin Pajankar. *Python Unit Test Automation: Automate, Organize, and Execute Unit Tests in Python*. Apress Media, LLC, Dez. 2021. ISBN: 978-1-4842-7854-3,
 - Alfredo Deza und Noah Gift. *Testing In Python*. Pragmatic AI Labs, Feb. 2020. ISBN: 979-8-6169-6064-1,
 - Moritz Lenz. *Python Continuous Integration and Delivery: A Concise Guide with Examples*. Apress Media, LLC, Dez. 2018. ISBN: 978-1-4842-4281-0,
 - Kristian Rother. *Pro Python Best Practices: Debugging, Testing and Maintenance*. Apress Media, LLC, März 2017. ISBN: 978-1-4842-2241-6.

Literatur



- Für diesen Kurs reicht unser Kursbuch¹¹³.
- Weitere Bücher zu Data Science (DS), numerische Berechnungen, Visualisierung, und AI:
 - Wes McKinney. *Python for Data Analysis*. 3. Aufl. O'Reilly Media, Inc., Aug. 2022. ISBN: 978-1-0981-0403-0,
 - Ashwin Pajankar. *Hands-on Matplotlib: Learn Plotting and Visualizations with Python 3*. Apress Media, LLC, Nov. 2021. ISBN: 978-1-4842-7410-1,
 - Joel Grus. *Data Science from Scratch: First Principles with Python*. 2. Aufl. O'Reilly Media, Inc., Mai 2019. ISBN: 978-1-4920-4113-9,
 - Robert Johansson. *Numerical Python: Scientific Computing and Data Science Applications with NumPy, SciPy and Matplotlib*. Apress Media, LLC, Dez. 2018. ISBN: 978-1-4842-4246-9.

Literatur



- Für diesen Kurs reicht unser Kursbuch¹¹³.
- Bücher zu weiteren Themen:
 - Aaron Maxwell. *What are f-strings in Python and how can I use them?* Infinite Skills Inc, Juni 2017. ISBN: 978-1-4919-9486-3,
 - Abhishek Ratan, Eric Chou, Pradeeban Kathiravelu und Dr. M.O. Faruque Sarker. *Python Network Programming*. Packt Publishing Ltd, Jan. 2019. ISBN: 978-1-78883-546-6.

Literatur



- Für diesen Kurs reicht unser Kursbuch¹¹³.
- Onlineressourcen:
 - *Python 3 Documentation*. Python Software Foundation (PSF), 2001–2025. URL: <https://docs.python.org/3>,
 - The PEP Editors. *Index of Python Enhancement Proposals (PEPs)*. Python Enhancement Proposal (PEP) 0. Python Software Foundation (PSF), 13. Juli 2000. URL: <https://peps.python.org>,
 - *Real Python Tutorials*. DevCademy Media Inc., 2021–2025. URL: <https://realpython.com>,
 - *W3Schools: Python Tutorials*. Refsnes Data AS, 1999–2025. URL: <https://www.w3schools.com/python>,
 - Trey Hunner. *Python Morsels*. Python Morsels, 2025. URL: <https://www.pythonmorsels.com>,
 - Florian Bruhin. *Python f-Strings*. Bruhin Software, 31. Mai 2023. URL: <https://fstring.help>,
 - Benjamin Dicken. *PyFlo – The Beginners Guide to Becoming a Python Programmer*. 2023. URL: <https://pyflo.net>.



Zusammenfassung



Zusammenfassung

- Programmieren ist das Schreiben von Quellcode für Computerprogramme.



Zusammenfassung

- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.



Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.

Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.
- Man muss auch die dazugehörigen Werkzeuge verstehen, die bewährten Praktiken, die Stilrichtlinien, man muss wissen, wie man Programme testet, wie man sie dokumentiert, und so weiter.

Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.
- Man muss auch die dazugehörigen Werkzeuge verstehen, die bewährten Praktiken, die Stilrichtlinien, man muss wissen, wie man Programme testet, wie man sie dokumentiert, und so weiter.
- Man muss die wichtigsten Komponenten der *Softwareentwicklung* verstehen.

Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.
- Man muss auch die dazugehörigen Werkzeuge verstehen, die bewährten Praktiken, die Stilrichtlinien, man muss wissen, wie man Programme testet, wie man sie dokumentiert, und so weiter.
- Man muss die wichtigsten Komponenten der *Softwareentwicklung* verstehen.
- Ich werde versuchen, Ihnen Programmieren zusammen mit mehrerer solcher Aspekte beizubringen.

Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.
- Man muss auch die dazugehörigen Werkzeuge verstehen, die bewährten Praktiken, die Stilrichtlinien, man muss wissen, wie man Programme testet, wie man sie dokumentiert, und so weiter.
- Man muss die wichtigsten Komponenten der *Softwareentwicklung* verstehen.
- Ich werde versuchen, Ihnen Programmieren zusammen mit mehrerer solcher Aspekte beizubringen.
- Wir nutzen die Programmiersprache Python

Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.
- Man muss auch die dazugehörigen Werkzeuge verstehen, die bewährten Praktiken, die Stilrichtlinien, man muss wissen, wie man Programme testet, wie man sie dokumentiert, und so weiter.
- Man muss die wichtigsten Komponenten der *Softwareentwicklung* verstehen.
- Ich werde versuchen, Ihnen Programmieren zusammen mit mehrerer solcher Aspekte beizubringen.
- Wir nutzen die Programmiersprache Python, weil sie einfach zu lernen ist

Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.
- Man muss auch die dazugehörigen Werkzeuge verstehen, die bewährten Praktiken, die Stilrichtlinien, man muss wissen, wie man Programme testet, wie man sie dokumentiert, und so weiter.
- Man muss die wichtigsten Komponenten der *Softwareentwicklung* verstehen.
- Ich werde versuchen, Ihnen Programmieren zusammen mit mehrerer solcher Aspekte beizubringen.
- Wir nutzen die Programmiersprache Python, weil sie einfach zu lernen ist, weit-verbreitet

Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.
- Man muss auch die dazugehörigen Werkzeuge verstehen, die bewährten Praktiken, die Stilrichtlinien, man muss wissen, wie man Programme testet, wie man sie dokumentiert, und so weiter.
- Man muss die wichtigsten Komponenten der *Softwareentwicklung* verstehen.
- Ich werde versuchen, Ihnen Programmieren zusammen mit mehrerer solcher Aspekte beizubringen.
- Wir nutzen die Programmiersprache Python, weil sie einfach zu lernen ist, weit-verbreitet, ein reiches Ökosystem von nützlichen Bibliotheken und Werkzeugen bietet

Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.
- Man muss auch die dazugehörigen Werkzeuge verstehen, die bewährten Praktiken, die Stilrichtlinien, man muss wissen, wie man Programme testet, wie man sie dokumentiert, und so weiter.
- Man muss die wichtigsten Komponenten der *Softwareentwicklung* verstehen.
- Ich werde versuchen, Ihnen Programmieren zusammen mit mehrerer solcher Aspekte beizubringen.
- Wir nutzen die Programmiersprache Python, weil sie einfach zu lernen ist, weit-verbreitet, ein reiches Ökosystem von nützlichen Bibliotheken und Werkzeugen bietet, und einen einfachen Buildprozess hat.

Zusammenfassung



- Programmieren ist das Schreiben von Quellcode für Computerprogramme.
- Wir können dafür eine Programmiersprache wie Python verwenden.
- Um gute, nützliche, und wartbare Programme zu schreiben, ist es nicht genug, eine Programmiersprache zu erlernen.
- Man muss auch die dazugehörigen Werkzeuge verstehen, die bewährten Praktiken, die Stilrichtlinien, man muss wissen, wie man Programme testet, wie man sie dokumentiert, und so weiter.
- Man muss die wichtigsten Komponenten der *Softwareentwicklung* verstehen.
- Ich werde versuchen, Ihnen Programmieren zusammen mit mehrerer solcher Aspekte beizubringen.
- Wir nutzen die Programmiersprache Python, weil sie einfach zu lernen ist, weit-verbreitet, ein reiches Ökosystem von nützlichen Bibliotheken und Werkzeugen bietet, und einen einfachen Buildprozess hat.
- Aber genug der Einleitung. Jetzt installieren wir erst mal Python!



谢谢你们！
Thank you!
Vielen Dank!



References I



- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Gordon Murray, Benoit Steiner, Paul A. Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu und Xiaoqiang Zheng. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI'2016)*. 2.–4. Nov. 2016, Savannah, GA, USA. Hrsg. von Kimberly Keeton und Timothy Roscoe. Berkeley, CA, USA: USENIX Association, 2016, S. 265–283. ISBN: 978-1-931971-33-1. URL: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi> (besucht am 2024-06-26) (siehe S. 71–80, 134).
- [2] David Lee Applegate, Robert E. Bixby, Vašek Chvátal und William John Cook. *The Traveling Salesman Problem: A Computational Study*. 2. Aufl. Bd. 17 der Reihe Princeton Series in Applied Mathematics. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: 978-0-691-12993-8 (siehe S. 134).
- [3] Thomas Bäck, David B. Fogel und Zbigniew „Zbyszek“ Michalewicz, Hrsg. *Handbook of Evolutionary Computation*. Bristol, England, UK: IOP Publishing Ltd und Oxford, Oxfordshire, England, UK: Oxford University Press, 1997. ISBN: 978-0-7503-0392-7 (siehe S. 130, 133).
- [4] Paul Barry. *Head First Python*. 3. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., Aug. 2023. ISBN: 978-1-4920-5129-9 (siehe S. 94, 95).
- [5] David M. Beazley. "Data Processing with Pandas". ;login: *Usenix Magazin* 37(6), Dez. 2012. Berkeley, CA, USA: USENIX Association. ISSN: 1044-6397. URL: <https://www.usenix.org/publications/login/december-2012-volume-37-number-6/data-processing-pandas> (besucht am 2024-06-25) (siehe S. 71–80, 132).
- [6] Kent L. Beck. *JUnit Pocket Guide*. Sebastopol, CA, USA: O'Reilly Media, Inc., Sep. 2004. ISBN: 978-0-596-00743-0 (siehe S. 134).
- [7] Tim Berners-Lee. *Re: Qualifiers on Hypertext links...* Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) und Newsgroups: alt.hypertext, 6. Aug. 1991. URL: <https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt> (besucht am 2025-02-05) (siehe S. 131, 135).
- [8] Fabian Beuke. *GitHut 2.0: GitHub Language Statistics*. San Francisco, CA, USA: GitHub Inc, 2023. URL: <https://madnight.github.io/github> (besucht am 2024-06-24) (siehe S. 71–80).

References II



- [9] Jacek Błażewicz, Wolfgang Domschke und Erwin Pesch. "The Job Shop Scheduling Problem: Conventional and New Solution Techniques". *European Journal of Operational Research* 93(1):1–33, Aug. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(95)00362-2 (siehe S. 131).
- [10] Florian Bruhin. *Python f-Strings*. Winterthur, Switzerland: Bruhin Software, 31. Mai 2023. URL: <https://fstring.help> (besucht am 2024-07-25) (siehe S. 99, 131).
- [11] Rainer E. Burkard, Eranda Çela, Panos Miltiades Pardalos und Leonidas S. Pitsoulis. "The Quadratic Assignment Problem". In: *Handbook of Combinatorial Optimization*. Hrsg. von Panos Miltiades Pardalos, Ding-Zhu Du und Ronald Lewis Graham. 1. Aufl. Boston, MA, USA: Springer, 1998, S. 1713–1809. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_27 (siehe S. 133).
- [12] Eduardo Carvalho Pinto und Carola Doerr. *Towards a More Practice-Aware Runtime Analysis of Evolutionary Algorithms*. arXiv.org: Computing Research Repository (CoRR) abs/1812.00493. Ithaca, NY, USA: Cornell University Library, 3. Dez. 2018. doi:10.48550/arXiv.1812.00493. URL: <https://arxiv.org/abs/1812.00493> (besucht am 2025-08-08). arXiv:1812.00493v1 [cs.NE] 3 Dec 2018 (siehe S. 130).
- [13] Oscar Castro, Pierrick Bruneau, Jean-Sébastien Sottet und Dario Torregrossa. "Landscape of High-Performance Python to Develop Data Science and Machine Learning Applications". *ACM Computing Surveys (CSUR)* 56(3):65:1–65:30, 2024. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0360-0300. doi:10.1145/3617588 (siehe S. 71–80).
- [14] Bo Chen, Chris N. Potts und Gerhard J. Woeginger. "A Review of Machine Scheduling: Complexity, Algorithms and Approximability". In: *Handbook of Combinatorial Optimization*. Hrsg. von Panos Miltiades Pardalos, Ding-Zhu Du und Ronald Lewis Graham. 1. Aufl. Boston, MA, USA: Springer, 1998, S. 1493–1641. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_25. See also pages 21–169 in volume 3/3 by Norwell, MA, USA: Kluwer Academic Publishers. (Siehe S. 131, 135).
- [15] Jiayang Chen (陈嘉阳), Zhize Wu (吴志泽), Sarah Louise Thomson und Thomas Weise (汤卫恩). "Frequency Fitness Assignment: Optimization Without Bias for Good Solution Outperforms Randomized Local Search on the Quadratic Assignment Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Hrsg. von Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, S. 27–37. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888600003837 (siehe S. 131–133).

References III



- [16] Philippe Chrétienne, Edward G. Coffman, Jan Karel Lenstra und Zhen Liu, Hrsg. *Scheduling Theory and Its Applications*. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Sep. 1995. ISBN: **978-0-471-94059-3** (siehe S. **131**).
- [17] Stephen Arthur Cook. "The Complexity of Theorem-Proving Procedures". In: *Third Annual ACM Symposium on Theory of Computing (STOC'1971)*. 3.–5. Mai 1971, Shaker Heights, OH, USA. Hrsg. von Michael A. Harrison, Ranan B. Banerji und Jeffrey D. Ullman. New York, NY, USA: Association for Computing Machinery (ACM), 1971, S. 151–158. ISBN: **978-1-4503-7464-4**. doi:**10.1145/800157.805047** (siehe S. **135**).
- [18] Kenneth Alan De Jong. *Evolutionary Computation: A Unified Approach*. Bd. 4 der Reihe Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, 2006. ISBN: **978-0-262-04194-2**. URL: <https://www.researchgate.net/publication/220740669> (besucht am 2025-08-08) (siehe S. **133**).
- [19] *Definition of Operations Research*. University of Western Ontario, London, ON, Canada: International Federation of Operational Research Societies (IFORS), 2020. URL: <https://www.ifors.org/what-is-or> (besucht am 2026-01-01) (siehe S. **132**).
- [20] Paul Deitel, Harvey Deitel und Abbey Deitel. *Internet & World Wide WebW[: How to Program*. 5. Aufl. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: **978-0-13-299045-5** (siehe S. **135**).
- [21] Justin Dennison, Cherokee Boose und Peter van Rysdam. *Intro to NumPy*. Centennial, CO, USA: ACI Learning. Birmingham, England, UK: Packt Publishing Ltd, Juni 2024. ISBN: **978-1-83620-863-1** (siehe S. **71–80, 132**).
- [22] *Developer Survey 2019: Open Source Runtime Pains*. Vancouver, BC, Canada: ActiveState Software Inc., 30. Apr. 2019. URL: <https://cdn.activestate.com/wp-content/uploads/2019/05/ActiveState-Developer-Survey-2019-Open-Source-Runtime-Pains.pdf> (besucht am 2024-12-29) (siehe S. **61–69**).
- [23] Alfredo Deza und Noah Gift. *Testing In Python*. San Francisco, CA, USA: Pragmatic AI Labs, Feb. 2020. ISBN: **979-8-6169-6064-1** (siehe S. **96, 132**).
- [24] Benjamin Dicken. *PyFlo – The Beginners Guide to Becoming a Python Programmer*. 2023. URL: <https://pyflo.net> (besucht am 2025-08-28) (siehe S. **99**).

References IV



- [25] Slobodan Dmitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, März 2024. ISBN: **979-8-8688-0224-9** (siehe S. **130**).
- [26] Stefan Droste, Thomas Jansen und Ingo Wegener. "On the Analysis of the $(1 + 1)$ Evolutionary Algorithm". *Theoretical Computer Science* 276(1-2):51–81, Apr. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: **0304-3975**. doi:**10.1016/S0304-3975(01)00182-7** (siehe S. **130**).
- [27] Kelly Easton, George L. Nemhauser und Michael A. Trick. "The Traveling Tournament Problem Description and Benchmarks". In: *7th International Conference on Principles and Practice of Constraint Programming (CP'01)*. 26. Nov.–1. Dez. 2001, Paphos, Cyprus. Hrsg. von Toby Walsh. Bd. 2239 der Reihe Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2001, S. 580–584. ISSN: **0302-9743**. ISBN: **978-3-540-42863-3**. doi:**10.1007/3-540-45578-7_43** (siehe S. **134**).
- [28] *ECMAScript Language Specification*. Standard ECMA-262, 3rd Edition. Geneva, Switzerland: Ecma International, Dez. 1999. URL: https://ecma-international.org/wp-content/uploads/ECMA-262_3rd_edition_december_1999.pdf (besucht am 2024-12-15) (siehe S. **131**).
- [29] "Formatted String Literals". In: *Python 3 Documentation. The Python Tutorial*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. Kap. 7.1.1. URL: <https://docs.python.org/3/tutorial/inputoutput.html#formatted-string-literals> (besucht am 2024-07-25) (siehe S. **131**).
- [30] Michael R. Garey und David Stifler Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman and Company, 1979. ISBN: **978-0-7167-1045-5** (siehe S. **134**).
- [31] Bhavesh Gawade. "Mastering F-Strings in Python: Efficient String Handling in Python Using Smart F-Strings". In: *C O D E B*. Mumbai, Maharashtra, India: Code B Solutions Pvt Ltd, 25. Apr.–3. Juni 2025. URL: <https://code-b.dev/blog/f-strings-in-python> (besucht am 2025-08-04) (siehe S. **131**).
- [32] GitHub Staff. *Octoverse: AI leads Python to top language as the number of global developers surges*. San Francisco, CA, USA: GitHub Inc, 29. Okt. 2024. URL: <https://github.blog/news-insights/octoverse/octoverse-2024> (besucht am 2025-01-07) (siehe S. **71–80**).

References V



- [33] David Edward Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Redwood City, CA, USA: Addison Wesley Longman Publishing Co., Inc., 1989. ISBN: **978-0-201-15767-3** (siehe S. **133**).
- [34] Michael T. Goodrich. *A Gentle Introduction to NP-Completeness*. Irvine, CA, USA: University of California, Irvine, Apr. 2022. URL: <https://ics.uci.edu/~goodrich/teach/cs165/notes/NPComplete.pdf> (besucht am 2025-08-01) (siehe S. **135**).
- [35] Michael Goodwin. *What is an API?* Armonk, NY, USA: International Business Machines Corporation (IBM), 9. Apr. 2024. URL: <https://www.ibm.com/topics/api> (besucht am 2024-12-12) (siehe S. **130**).
- [36] Olaf Górski. "Why f-strings are awesome: Performance of different string concatenation methods in Python". In: *DEV Community*. Sacramento, CA, USA: DEV Community Inc., 8. Nov. 2022. URL: <https://dev.to/grski/performance-of-different-string-concatenation-methods-in-python-why-f-strings-are-awesome-2e97> (besucht am 2025-08-04) (siehe S. **131**).
- [37] Linda Grandell, Mia Peltomäki, Ralph-Johan Back und Tapio Salakoski. "Why complicate things? Introducing Programming in High School using Python". In: *8th Australasian Conference on Computing Education (ACE'2006)*. 16.–19. Jan. 2006, Hobart, TAS, Australia. Hrsg. von Denise Tolhurst und Samuel Mann. Bd. 52. New York, NY, USA: Association for Computing Machinery (ACM), 2006, S. 71–80. ISBN: **978-1-920682-34-7**. doi:[10.5555/1151869.1151880](https://doi.org/10.5555/1151869.1151880) (siehe S. **71–80**).
- [38] Joel Grus. *Data Science from Scratch: First Principles with Python*. 2. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., Mai 2019. ISBN: **978-1-4920-4113-9** (siehe S. **71–80, 97, 130**).
- [39] Gregory Z. Gutin und Abraham P. Punnen, Hrsg. *The Traveling Salesman Problem and its Variations*. Bd. 12 der Reihe Combinatorial Optimization (COOP). New York, NY, USA: Springer New York, Mai 2002. ISSN: **1388-3011**. doi:[10.1007/b101971](https://doi.org/10.1007/b101971) (siehe S. **134**).
- [40] Charles R. Harris, K. Jarrod Millman, Stéfan van der Walt, Ralf Gommers, Pauli „pv“ Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Pícus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke und Travis E. Oliphant. "Array programming with NumPy". *Nature* 585:357–362, 2020. London, England, UK: Springer Nature Limited. ISSN: **0028-0836**. doi:[10.1038/S41586-020-2649-2](https://doi.org/10.1038/S41586-020-2649-2) (siehe S. **71–80, 132**).

References VI



- [41] Ian Hickson, Robin Berjon, Steve Faulkner, Travis Leithead, Erika Doyle Navara, Theresa O'Connor und Silvia Pfeiffer, Hrsg. *HTML5: A Vocabulary and Associated APIs for HTML and XHTML*. W3C Recommendation. Wakefield, MA, USA: World Wide Web Consortium (W3C), 28. Okt. 2014. URL: <http://www.w3.org/TR/2014/REC-html5-20141028> (besucht am 2024-12-17) (siehe S. 131).
- [42] Trey Hunner. *Python Morsels*. Reykjavík, Iceland: Python Morsels, 2025. URL: <https://www.pythonmorsels.com> (besucht am 2025-04-17) (siehe S. 99).
- [43] John Hunt. *A Beginners Guide to Python 3 Programming*. 2. Aufl. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (siehe S. 94, 95, 132).
- [44] John D. Hunter. "Matplotlib: A 2D Graphics Environment". *Computing in Science & Engineering* 9(3):90–95, Mai–Juni 2007. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1521-9615. doi:10.1109/MCSE.2007.55 (siehe S. 71–80, 131).
- [45] John D. Hunter, Darren Dale, Eric Firing, Michael Droettboom und The Matplotlib Development Team. *Matplotlib: Visualization with Python*. Austin, TX, USA: NumFOCUS, Inc., 2012–2025. URL: <https://matplotlib.org> (besucht am 2025-02-02) (siehe S. 131).
- [46] Mike James. *Programmer's Python: Everything is an Object – Something Completely Different*. 2. Aufl. I/O Press, 25. Juni 2022 (siehe S. 94, 95).
- [47] Robert Johansson. *Numerical Python: Scientific Computing and Data Science Applications with NumPy, SciPy and Matplotlib*. New York, NY, USA: Apress Media, LLC, Dez. 2018. ISBN: 978-1-4842-4246-9 (siehe S. 71–80, 97, 131–133).
- [48] Stephen Curtis Johnson. *Lint, a C Program Checker*. Computing Science Technical Report 78-1273. New York, NY, USA: Bell Telephone Laboratories, Incorporated, 25. Okt. 1978. URL: <https://wolfram.schneider.org/bsd/7thEdManVol2/lint/lint.pdf> (besucht am 2024-08-23) (siehe S. 131).
- [49] Bernd Klein. *Einführung in Python 3 – Für Ein- und Umsteiger*. 3., überarbeitete. München, Bayern, Germany: Carl Hanser Verlag GmbH & Co. KG, 2018. ISBN: 978-3-446-45208-4. doi:10.3139/9783446453876 (siehe S. 94, 95).

References VII



- [50] Tjalling Charles Koopmans und Martin Beckmann. "Assignment Problems and the Location of Economic Activities". *Econometrica* 25(1):53–76, 1957. New Haven, CT, USA: The Econometric Society and Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0012-9682. doi:10.2307/1907742 (siehe S. 133).
- [51] Holger Krekel und pytest-Dev Team. *pytest Documentation*. Release 8.4. Freiburg, Baden-Württemberg, Germany: merlinux GmbH. URL: <https://readthedocs.org/projects/pytest/downloads/pdf/latest> (besucht am 2024-11-07) (siehe S. 132).
- [52] Charles Landau. *TensorFlow Deep Dive: Build, Train, and Deploy Machine Learning Models with TensorFlow*. Sebastopol, CA, USA: O'Reilly Media, Inc., Dez. 2023 (siehe S. 71–80, 134).
- [53] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan und David B. Shmoys. "Sequencing and Scheduling: Algorithms and Complexity". In: *Production Planning and Inventory*. Hrsg. von Stephen C. Graves, Alexander Hendrik George Rinnooy Kan und Paul H. Zipkin. Bd. IV der Reihe Handbooks of Operations Research and Management Science. Amsterdam, The Netherlands: Elsevier B.V., 1993. Kap. 9, S. 445–522. ISSN: 0927-0507. ISBN: 978-0-444-87472-6. doi:10.1016/S0927-0507(05)80189-6. URL: <http://alexandria.tue.nl/repository/books/339776.pdf> (besucht am 2023-12-06) (siehe S. 131, 135).
- [54] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan und David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Estimation, Simulation, and Control – Wiley-Interscience Series in Discrete Mathematics and Optimization. Chichester, West Sussex, England, UK: Wiley Interscience, Sep. 1985. ISSN: 0277-2698. ISBN: 978-0-471-90413-7 (siehe S. 134).
- [55] Kent D. Lee und Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (siehe S. 132).
- [56] Moritz Lenz. *Python Continuous Integration and Delivery: A Concise Guide with Examples*. New York, NY, USA: Apress Media, LLC, Dez. 2018. ISBN: 978-1-4842-4281-0 (siehe S. 96, 130).
- [57] Reuven M. Lerner. *Pandas Workout*. Shelter Island, NY, USA: Manning Publications, Juni 2024. ISBN: 978-1-61729-972-8 (siehe S. 71–80, 132).

References VIII



- [58] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, Sarah Louise Thomson und Thomas Weise (汤卫思). "Addressing the Traveling Salesperson Problem with Frequency Fitness Assignment and Hybrid Algorithms". *Soft Computing* 28(17-18):9495–9508, Juli 2024. London, England, UK: Springer Nature Limited. ISSN: **1432-7643**. doi:[10.1007/S00500-024-09718-8](https://doi.org/10.1007/S00500-024-09718-8) (siehe S. **132**, **134**).
- [59] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg und Thomas Weise (汤卫思). "Solving the Traveling Salesperson Problem using Frequency Fitness Assignment". In: *IEEE Symposium Series on Computational Intelligence (SSCI'2022)*. 4.–7. Dez. 2022, Singapore. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022. ISBN: **978-1-6654-8769-6**. doi:[10.1109/SSCI51031.2022.10022296](https://doi.org/10.1109/SSCI51031.2022.10022296) (siehe S. **132**).
- [60] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Matthias Thüerer, Markus Wagner und Thomas Weise (汤卫思). "Generating Small Instances with Interesting Features for the Traveling Salesperson Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Hrsg. von Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, S. 173–180. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0012888800003837](https://doi.org/10.5220/0012888800003837) (siehe S. **132**).
- [61] Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter M. Hahn und Tania Querido. "A Survey for the Quadratic Assignment Problem". *European Journal of Operational Research* 176(2):657–690, 2007. Amsterdam, The Netherlands: Elsevier B.V. ISSN: **0377-2217**. doi:[10.1016/j.ejor.2005.09.032](https://doi.org/10.1016/j.ejor.2005.09.032) (siehe S. **133**).
- [62] Mark Lutz. *Learning Python*. 6. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., März 2025. ISBN: **978-1-0981-7130-8** (siehe S. **94**, **95**, **132**).
- [63] *Making Open Source Work Better for Developers: Highlights of the 2019 Tidelift Managed Open Source Survey*. Boston, MA, USA: Tidelift, Inc., Juni 2019. URL: <https://tidelift.com/subscription/managed-open-source-survey> (besucht am 2024-12-29) (siehe S. **61–69**).
- [64] Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe, Hrsg. *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0000195000003837](https://doi.org/10.5220/0000195000003837).
- [65] Charlie Marsh. "Ruff". In: URL: <https://pypi.org/project/ruff> (besucht am 2025-08-29) (siehe S. **133**).

References IX



- [66] Charlie Marsh. *ruff: An Extremely Fast Python Linter and Code Formatter, Written in Rust*. New York, NY, USA: Astral Software Inc., 28. Aug. 2022. URL: <https://docs.astral.sh/ruff> (besucht am 2024-08-23) (siehe S. 133).
- [67] Eric Matthes. *Python Crash Course*. 3. Aufl. San Francisco, CA, USA: No Starch Press, Jan. 2023. ISBN: 978-1-7185-0270-3 (siehe S. 94, 95).
- [68] Aaron Maxwell. *What are f-strings in Python and how can I use them?* Oakville, ON, Canada: Infinite Skills Inc, Juni 2017. ISBN: 978-1-4919-9486-3 (siehe S. 98, 131).
- [69] Wes McKinney. *Python for Data Analysis*. 3. Aufl. Sebastopol, CA, USA: O'Reilly Media, Inc., Aug. 2022. ISBN: 978-1-0981-0403-0 (siehe S. 97).
- [70] Pedro Mejia Alvarez, Raul E. Gonzalez Torres und Susana Ortega Cisneros. *Exception Handling – Fundamentals and Programming*. SpringerBriefs in Computer Science. Cham, Switzerland: Springer, Feb. 2024. ISSN: 2191-5768. ISBN: 978-3-031-50680-2. doi:10.1007/978-3-031-50681-9 (siehe S. 61–69).
- [71] Zbigniew „Zbyszek“ Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1996. ISBN: 978-3-540-58090-4. doi:10.1007/978-3-662-03315-9 (siehe S. 133).
- [72] Melanie Mitchell. *An Introduction to Genetic Algorithms*. Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, Feb. 1998. ISBN: 978-0-262-13316-6. URL: <http://boente.eti.br/fuzzy/ebook-fuzzy-mitchell.pdf> (besucht am 2025-08-08) (siehe S. 133).
- [73] NumPy Team. *NumPy*. San Francisco, CA, USA: GitHub Inc und Austin, TX, USA: NumFOCUS, Inc. URL: <https://numpy.org> (besucht am 2025-02-02) (siehe S. 132).
- [74] A. Jefferson Offutt. “Unit Testing Versus Integration Testing”. In: *Test: Faster, Better, Sooner – IEEE International Test Conference (ITC'1991)*. 26.–30. Okt. 1991, Nashville, TN, USA. Los Alamitos, CA, USA: IEEE Computer Society, 1991. Kap. Paper P2.3, S. 1108–1109. ISSN: 1089-3539. ISBN: 978-0-8186-9156-0. doi:10.1109/TEST.1991.519784 (siehe S. 134).
- [75] Brian Okken. *Python Testing with pytest*. Flower Mound, TX, USA: Pragmatic Bookshelf by The Pragmatic Programmers, L.L.C., Feb. 2022. ISBN: 978-1-68050-860-4 (siehe S. 96, 132).

References X



- [76] Michael Olan. "Unit Testing: Test Early, Test Often". *Journal of Computing Sciences in Colleges (JCSC)* 19(2):319–328, Dez. 2003. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: **1937-4771**. doi:**10.5555/948785.948830**. URL: <https://www.researchgate.net/publication/255673967> (besucht am 2025-09-05) (siehe S. **134**).
- [77] Ashwin Pajankar. *Hands-on Matplotlib: Learn Plotting and Visualizations with Python 3*. New York, NY, USA: Apress Media, LLC, Nov. 2021. ISBN: **978-1-4842-7410-1** (siehe S. **71–80, 97, 131**).
- [78] Ashwin Pajankar. *Python Unit Test Automation: Automate, Organize, and Execute Unit Tests in Python*. New York, NY, USA: Apress Media, LLC, Dez. 2021. ISBN: **978-1-4842-7854-3** (siehe S. **96, 132, 134**).
- [79] Pandas Developers. *Pandas*. Austin, TX, USA: NumFOCUS, Inc. und Montreal, QC, Canada: OVHcloud. URL: <https://pandas.pydata.org> (besucht am 2025-02-02) (siehe S. **132**).
- [80] Panos Miltiades Pardalos, Ding-Zhu Du und Ronald Lewis Graham, Hrsg. *Handbook of Combinatorial Optimization*. 1. Aufl. Boston, MA, USA: Springer, 1998. ISBN: **978-1-4613-7987-4**.
- [81] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai und Soumith Chintala. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems (NeurIPS'2019)*. 8.–14. Dez. 2019, Vancouver, BC, Canada. Hrsg. von Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox und Roman Garnett. San Diego, CA, USA: The Neural Information Processing Systems Foundation (NeurIPS), 2019, S. 8024–8035. ISBN: **978-1-7138-0793-3**. URL: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html> (besucht am 2024-07-18) (siehe S. **71–80, 132**).
- [82] Fabian Pedregos, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot und Edouard Duchesnay. "Scikit-learn: Machine Learning in Python". *Journal of Machine Learning Research (JMLR)* 12:2825–2830, Okt. 2011. Cambridge, MA, USA: MIT Press. ISSN: **1532-4435**. doi:**10.5555/1953048.2078195** (siehe S. **71–80, 133**).

References XI



- [83] Yasset Pérez-Riverol, Laurent Gatto, Rui Wang, Timo Sachsenberg, Julian Uszkoreit, Felipe da Veiga Leprevost, Christian Fufezan, Tobias Ternent, Stephen J. Egle, Daniel S. Katz, Tom J. Pollard, Alexander Kononov, Robert M. Flight, Kai Blin und Juan Antonio Vizcaino. "Ten Simple Rules for Taking Advantage of Git and GitHub". *PLOS Computational Biology* 12(7), 14. Juli 2016. San Francisco, CA, USA: Public Library of Science (PLOS). ISSN: **1553-7358**. doi:[10.1371/JOURNAL.PCBI.1004947](https://doi.org/10.1371/JOURNAL.PCBI.1004947) (siehe S. **131**).
- [84] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/31EC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) und International Electrotechnical Commission (IEC), Nov. 2017. URL: <https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf> (besucht am 2024-06-29) (siehe S. **130**).
- [85] "programming: Meaning of *programming* in English". In: *Cambridge Dictionary English (UK)*. Cambridge, England, UK: Cambridge University Press & Assessment, Juni 2024. URL: <https://dictionary.cambridge.org/dictionary/english/programming> (besucht am 2024-06-17) (siehe S. **20, 21**).
- [86] *Python 3 Documentation*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2025. URL: <https://docs.python.org/3> (besucht am 2024-07-05) (siehe S. **99**).
- [87] Sanatan Rai und George Vairaktarakis. "NP-Complete Problems and Proof Methodology". In: *Encyclopedia of Optimization*. Hrsg. von Christodoulos A. Floudas und Panos Miliadis Pardalos. 2. Aufl. Boston, MA, USA: Springer, Sep. 2008, S. 2675–2682. ISBN: **978-0-387-74758-3**. doi:[10.1007/978-0-387-74759-0_462](https://doi.org/10.1007/978-0-387-74759-0_462) (siehe S. **135**).
- [88] Sebastian Raschka, Yuxi Liu und Vahid Mirjalili. *Machine Learning with PyTorch and Scikit-learn*. Birmingham, England, UK: Packt Publishing Ltd, Feb. 2022. ISBN: **978-1-80181-931-2** (siehe S. **71–80, 132, 133**).
- [89] Abhishek Ratan, Eric Chou, Pradeeban Kathiravelu und Dr. M.O. Faruque Sarker. *Python Network Programming*. Birmingham, England, UK: Packt Publishing Ltd, Jan. 2019. ISBN: **978-1-78883-546-6** (siehe S. **98**).
- [90] *Real Python Tutorials*. Vancouver, BC, Canada: DevCademy Media Inc., 2021–2025. URL: <https://realpython.com> (besucht am 2025-04-17) (siehe S. **99**).
- [91] Kristian Rother. *Pro Python Best Practices: Debugging, Testing and Maintenance*. New York, NY, USA: Apress Media, LLC, März 2017. ISBN: **978-1-4842-2241-6** (siehe S. **96**).

References XII



- [92] Per Runeson. "A Survey of Unit Testing Practices". *IEEE Software* 23(4):22–29, Juli–Aug. 2006. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0740-7459. doi:10.1109/MS.2006.91 (siehe S. 134).
- [93] Stuart J. Russell und Peter Norvig. *Artificial Intelligence: A Modern Approach (AIMA)*. 4. Aufl. Hoboken, NJ, USA: Pearson Education, Inc. ISBN: 978-1-292-40113-3. URL: <https://aima.cs.berkeley.edu> (besucht am 2024-06-27) (siehe S. 71–80, 130).
- [94] Yeonhee Ryou, Sangwoo Joh, Joonmo Yang, Sujin Kim und Youil Kim. "Code Understanding Linter to Detect Variable Misuse". In: *37th IEEE/ACM International Conference on Automated Software Engineering (ASE'2022)*. 10.–14. Okt. 2022, Rochester, MI, USA. New York, NY, USA: Association for Computing Machinery (ACM), 2022, 133:1–133:5. ISBN: 978-1-4503-9475-8. doi:10.1145/3551349.3559497 (siehe S. 131).
- [95] Sartaj Sahni und Teofilo Francisco Gonzalez Arce. "P-complete Approximation Problems". *Journal of the ACM (JACM)* 23(3):555–565, Juli 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0004-5411. doi:10.1145/321958.321975. URL: <https://sites.cs.ucsb.edu/~teo/papers/JACM-PCOM.pdf> (besucht am 2026-02-21) (siehe S. 133).
- [96] Shai Shalev-Shwartz und Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge, England, UK: Cambridge University Press (CUP), Juli 2014. ISBN: 978-1-107-05713-5. URL: <http://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning> (besucht am 2024-06-27) (siehe S. 71–80, 132).
- [97] Anna Skoulikari. *Learning Git*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mai 2023. ISBN: 978-1-0981-3391-7 (siehe S. 131).
- [98] Brett Slatkin. *Effective Python: 125 Specific Ways to Write Better Python*. 3. Aufl. Reading, MA, USA: Addison-Wesley Professional, Nov. 2024. ISBN: 978-0-13-817239-8 (siehe S. 94, 95).
- [99] Eric V. „ericvsmith“ Smith. *Literal String Interpolation*. Python Enhancement Proposal (PEP) 498. Beaverton, OR, USA: Python Software Foundation (PSF), 6. Nov. 2016–9. Sep. 2023. URL: <https://peps.python.org/pep-0498> (besucht am 2024-07-25) (siehe S. 131).
- [100] "Stack Overflow 2024 Developer Survey". In: *Stack Overflow*. New York, NY, USA: Stack Exchange Inc., Mai–Juni 2024. URL: <https://survey.stackoverflow.co/2024> (besucht am 2025-06-01) (siehe S. 71–80).

References XIII



- [101] Éric D. Taillard. "Benchmarks for Basic Scheduling Problems". *European Journal of Operational Research* 64(2):278–285, Jan. 1993. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(93)90182-M. URL: <https://arodes.hes-so.ch/record/8224/files/Preprint.pdf> (besucht am 2026-02-20) (siehe S. 131).
- [102] The PEP Editors. *Index of Python Enhancement Proposals (PEPs)*. Python Enhancement Proposal (PEP) 0. Beaverton, OR, USA: Python Software Foundation (PSF), 13. Juli 2000. URL: <https://peps.python.org> (besucht am 2025-04-17) (siehe S. 99).
- [103] George K. Thiruvathukal, Konstantin Läufer und Benjamin Gonzalez. "Unit Testing Considered Useful". *Computing in Science & Engineering* 8(6):76–87, Nov.–Dez. 2006. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1521-9615. doi:10.1109/MCSE.2006.124. URL: <https://www.researchgate.net/publication/220094077> (besucht am 2024-10-01) (siehe S. 134).
- [104] Sarah Louise Thomson, Gabriela Ochoa, Daan van den Berg, Tianyu Liang (梁天宇) und Thomas Weise (汤卫恩). "Entropy, Search Trajectories, and Explainability for Frequency Fitness Assignment". In: *Parallel Problem Solving from Nature (PPSN XVIII)*. Bd. 1. 14.–18. Sep. 2024, Hagenberg, Mühlkreis, Austria. Hrsg. von Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado und Thomas Bäck. Bd. 15148 der Reihe Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer. ISSN: 0302-9743. ISBN: 978-3-031-70054-5. doi:10.1007/978-3-031-70055-2_23 (siehe S. 132).
- [105] Brad Traversy. *Modern HTML & CSS From The Beginning 2.0*. 2. Aufl. Birmingham, England, UK: Packt Publishing Ltd, Juli 2024. ISBN: 978-1-83588-056-2 (siehe S. 131).
- [106] Mariot Tsitoara. *Beginning Git and GitHub: Version Control, Project Management and Teamwork for the New Developer*. New York, NY, USA: Apress Media, LLC, März 2024. ISBN: 979-8-8688-0215-7 (siehe S. 131, 134).
- [107] Guido van Rossum. *Computer Programming for Everybody (Revised Proposal)*. A Scouting Expedition for the Programmers of Tomorrow. CNRI Proposal 90120-1a. Reston, VA, USA: Corporation for National Research Initiatives (CNRI), Juli 1999. URL: <https://www.python.org/doc/essays/cp4e> (besucht am 2024-06-27) (siehe S. 71–80).
- [108] Guido van Rossum, Barry Warsaw und Alyssa Coghlan. *Style Guide for Python Code*. Python Enhancement Proposal (PEP) 8. Beaverton, OR, USA: Python Software Foundation (PSF), 5. Juli 2001. URL: <https://peps.python.org/pep-0008> (besucht am 2024-07-27) (siehe S. 33–49).

References XIV



- [109] Kristian Verduin, Sarah Louise Thomson und Daan van den Berg. "Too Constrained for Genetic Algorithms. Too Hard for Evolutionary Computing. The Traveling Tournament Problem". In: *15th International Joint Conference on Computational Intelligence (IJCCI'23)*. 13.–15. Nov. 2023, Rome, Italy. Hrsg. von Niki van Stein, Francesco Marcelloni, H. K. Lam, Marie Cottrell und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2023, S. 246–257. ISSN: **2184-3236**. ISBN: **978-989-758-674-3**. doi:[10.5220/0012192100003595](https://doi.org/10.5220/0012192100003595) (siehe S. **134**).
- [110] Pauli „pv“ Virtanen, Ralf Gommers, Travis E. Oliphant, Matt „mdhaber“ Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, CJ Carey, Ilhan „ilayn“ Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregos, Paul van Mulbregt und SciPy 1.0 Contributors. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". *Nature Methods* 17:261–272, 2. März 2020. London, England, UK: Springer Nature Limited. ISSN: **1548-7091**. doi:[10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2). URL: <http://arxiv.org/abs/1907.10121> (besucht am 2024-06-26). See also arXiv:1907.10121v1 [cs.MS] 23 Jul 2019. (Siehe S. **71–80**, **133**).
- [111] *W3Schools: Python Tutorials*. Sandnes, Rogaland, Norway: Refsnes Data AS, 1999–2025. URL: <https://www.w3schools.com/python> (besucht am 2025-04-17) (siehe S. **99**).
- [112] Thomas Weise (汤卫思). *Global Optimization Algorithms – Theory and Application*. self-published, 2009. URL: <https://www.researchgate.net/publication/200622167> (besucht am 2025-07-25) (siehe S. **130**, **133**).
- [113] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2024–2025. URL: <https://thomasweise.github.io/programmingWithPython> (besucht am 2025-01-05) (siehe S. **94–99**, **132**, **133**).
- [114] Thomas Weise (汤卫思), Raymond Chiong, Jörg Lässig, Ke Tang (唐珂), Shigeyoshi Tsutsui, Wenxiang Chen (陈文祥), Zbigniew „Zbyszek“ Michalewicz und Xin Yao (姚新). "Benchmarking Optimization Algorithms: An Open Source Framework for the Traveling Salesman Problem". *IEEE Computational Intelligence Magazine (CIM)* 9(3):40–52, Aug. 2014. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: **1556-603X**. doi:[10.1109/MCI.2014.2326101](https://doi.org/10.1109/MCI.2014.2326101) (siehe S. **134**).

References XV



- [115] Thomas Weise (汤卫恩) und Zhize Wu (吴志泽). "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *Conference on Genetic and Evolutionary Computation (GECCO'2023), Companion Volume*. 15.–19. Juli 2023, Lisbon, Portugal. Hrsg. von Sara Silva und Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, S. 1891–1899. ISBN: **979-8-4007-0120-7**. doi:[10.1145/3583133.3596306](https://doi.org/10.1145/3583133.3596306) (siehe S. **71–80, 132**).
- [116] Thomas Weise (汤卫恩), Yuezhong Wu (吴越钟), Raymond Chiong, Ke Tang (唐珂) und Jörg Lässig. "Global versus Local Search: The Impact of Population Sizes on Evolutionary Algorithm Performance". *Journal of Global Optimization* 66(3):511–534, Feb. 2016. London, England, UK: Springer Nature Limited. ISSN: **0925-5001**. doi:[10.1007/s10898-016-0417-5](https://doi.org/10.1007/s10898-016-0417-5) (siehe S. **134**).
- [117] L. Darrell Whitley. "The GENITOR Algorithm and Selection Pressure: Why Rank-Based Allocation of Reproductive Trials is Best". In: *3rd International Conference on Genetic Algorithms (ICGA'1989)*. Juni 1989, Fairfax, VA, USA: George Mason University. Hrsg. von J. David Schaffer. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1989, S. 116–123. ISBN: **978-1-55860-066-9**. URL: <https://www.researchgate.net/publication/2527551> (besucht am 2025-08-08) (siehe S. **133**).
- [118] Kevin Wilson. *Python Made Easy*. Birmingham, England, UK: Packt Publishing Ltd, Aug. 2024. ISBN: **978-1-83664-615-0** (siehe S. **94, 95, 132**).
- [119] CAO Xiang (曹翔), Zhize Wu (吴志泽), Daan van den Berg und Thomas Weise (汤卫恩). "Randomized Local Search vs. NSGA-II vs. Frequency Fitness Assignment on The Traveling Tournament Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Hrsg. von Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, S. 38–49. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0012891500003837](https://doi.org/10.5220/0012891500003837) (siehe S. **132, 134**).
- [120] Rui Zhao (赵睿), Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer und Thomas Weise (汤卫恩). "Randomized Local Search on the 2D Rectangular Bin Packing Problem with Item Rotation". In: *Genetic and Evolutionary Computation Conference (GECCO'2024)*. 14.–18. Juli 2024, Melbourne, VIC, Australia. Hrsg. von Xiaodong Li und Julia Handl. New York, NY, USA: Association for Computing Machinery (ACM), 2024, S. 235–238. ISBN: **979-8-4007-0494-9**. doi:[10.1145/3638530.3654139](https://doi.org/10.1145/3638530.3654139) (siehe S. **132**).

References XVI



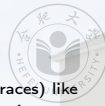
- [121] Rui Zhao (赵睿), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer, Tianyu Liang (梁天宇), Ming Tan (檀明) und Thomas Weise (汤卫思). "Randomized Local Search for Two-Dimensional Bin Packing and a Negative Result for Frequency Fitness Assignment". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. 20.–22. Nov. 2024, Porto, Portugal. Hrsg. von Francesco Marcelloni, Kurosh Madani, Niki van Stein und Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, S. 15–26. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0012888500003837](https://doi.org/10.5220/0012888500003837) (siehe S. **132**).
- [122] Dmitry Zinoviev. *Discrete Event Simulation: It's Easy with SimPy!* arXiv.org: Computing Research Repository (CoRR) abs/2405.01562. Ithaca, NY, USA: Cornell University Library, 3. Apr. 2024. doi:[10.48550/ARXIV.2405.01562](https://doi.org/10.48550/ARXIV.2405.01562). URL: <https://arxiv.org/abs/2405.01562> (besucht am 2024-06-27). arXiv:2405.01562v1 [cs.MS] 3 Apr 2024 (siehe S. **71–80**, **133**).

Glossary (in English) I



- (1 + 1) EA The (1 + 1) EA is a local search algorithm that retains the best solution x_c discovered so far during the search^{12,26}. In each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then the (1 + 1) EA uses a unary search operator that flips each bit independently with probability m/n , where usually $m = 1$. This operator is the main difference to randomized local search (RLS). The (1 + 1) EA is a special case of the $(\mu + \lambda)$ evolutionary algorithm $((\mu + \lambda)$ EA) where $\mu = \lambda = 1$.
- EA An *evolutionary algorithm* is a metaheuristic optimization method that maintains a population of candidate solutions, which undergo selection (where better solutions are chosen with higher probability) and reproduction (where mutation and recombination create a new candidate solution from one or two existing ones, respectively)^{3,112}.
- $(\mu + \lambda)$ EA The $(\mu + \lambda)$ EA is an evolutionary algorithm (EA) where, in each generation, λ offspring solutions are generated from the current population of μ parent solutions. The offspring and parent populations are merged, yielding $\mu + \lambda$ solutions, from which then the best μ solutions are retained to form the parent population of the next generation. If the search space is the bit strings of length n , then this algorithm usually applies a mutation operator flipping each bit independently with probability $1/n$.
- AI Artificial Intelligence, see, e.g.,⁹³
- API An *Application Programming Interface* is a set of rules or protocols that enables one software application or component to use or communicate with another³⁵.
- C is a programming language, which is very successful in system programming situations^{25,84}.
- CI *Continuous Integration* is a software development process where developers integrate new code into a codebase hosted in a Version Control Systems (VCS), after which automated tools run an automated build process including code analysis (such as linters) and unit test execution⁵⁶. If the build succeeds and no errors or problems with the code are, the code may automatically be deployed (if the CI system is configured to do so).
- DS Data Science, see, e.g.,³⁸.

Glossary (in English) II



- f-string** let you include the results of expressions in strings^{10,29,31,36,68,99}. They can contain expressions (in curly braces) like `f"a{6-1}b"` that are then transformed to text via (string) interpolation, which turns the string to `"a5b"`. F-strings are delimited by `f"..."`.
- FFA** *Frequency Fitness Assignment* is a algorithm plugin for optimization methods applied to discrete or combinatorial problems with not-too-many different possible objective values. It replaces the objective values in all comparisons with their absolute encounter frequency so far during the search. FFA has successfully been applied to the Quadratic Assignment Problem (QAP)¹⁵.
- Git** is a distributed Version Control Systems (VCS) which allows multiple users to work on the same code while preserving the history of the code changes^{97,106}. Learn more at <https://git-scm.com>.
- GitHub** is a website where software projects can be hosted and managed via the Git VCS^{83,106}. Learn more at <https://github.com>.
- HTML** The Hyper Text Markup Language (HTML) is the text format used by the World Wide Web (WWW)^{7,41,105}.
- JavaScript** JavaScript is the predominant programming language used in websites to develop interactive contents for display in browsers²⁸.
- JSSP** The *Job Shop Scheduling Problem*^{9,16,53} is one of the most prominent and well-studied scheduling tasks. In a JSSP instance, there are k machines and m jobs¹⁰¹. Each job must be processed once by each machine in a job-specific sequence and has a job-specific processing time on each machine. The goal is to find an assignment of jobs to machines that results in an overall shortest makespan, i.e., the schedule which can complete all the jobs in the shortest time. The JSSP is $\mathcal{NP}$ -complete^{14,53}.
- linter** A linter is a tool for analyzing program code to identify bugs, problems, vulnerabilities, and inconsistent code styles^{48,94}. Ruff is an example for a linter used in the Python world.
- Matplotlib** is a Python package for plotting diagrams and charts^{44,45,47,77}. Learn more at <https://matplotlib.org>⁴⁵.

Glossary (in English) III



ML Machine Learning, see, e.g.,⁹⁶

moptipy is the *Metaheuristic Optimization in Python* library¹¹⁵. It has been used in several different research works, including^{15,58–60,104,119–121}. Learn more at <https://thomasweise.github.io/moptipy> and <https://thomasweise.github.io/moptipyapps>.

NumPy is a fundamental package for scientific computing with Python, which offers efficient array datastructures^{21,40,47}. Learn more at <https://numpy.org>⁷³.

OR Operations Research (or Operational Research) is the application of sciences such as mathematics and computer science to the management and organization of systems, organizations, enterprises, factories, or projects. It encompasses the development and application of problem-solving methods and techniques (such as mathematical optimization, simulation, queueing theory and other stochastic models) with the goal to improve decision-making and efficiency¹⁹.

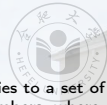
Pandas is a Python data analysis and manipulation library^{5,57}. Learn more at <https://pandas.pydata.org>⁷⁹.

pytest is a framework for writing and executing unit tests in Python^{23,51,75,78,118}. Learn more at <https://pytest.org>.

Python The Python programming language^{43,55,62,113}, i.e., what you will learn about in our book¹¹³. Learn more at <https://python.org>.

PyTorch is a Python library for deep learning and AI^{81,88}. Learn more at <https://pytorch.org>.

Glossary (in English) IV



QAP The *Quadratic Assignment Problem* is an optimization problem where the goal is to assign a set of n facilities to a set of n locations^{11,15,50,61}. Such an assignment can be represented as a permutation x of the first n natural numbers, where x_i specifies the location where facility i should be placed. For each QAP, a distance matrix D is given, where D_{pq} specifies the distance from location p to location q , as well as a flow matrix F , where F_{ij} is the amount of material flowing from facility i to facility j . The objective function f then rates a permutation x as $f(x) = \sum_{i=1}^n \sum_{j=1}^n D_{x_i x_j} F_{ij}$. The QAP is $\mathcal{NP}$ -complete⁹⁵.

RLS Randomized local search retains the best solution x_c discovered so far during the search and, in each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then RLS uses a unary search operator that flips exactly one bit. This operator is the main difference to $(1+1)$ evolutionary algorithm $((1+1) \text{ EA})$.

Ruff is a linter and code formatting tool for Python^{65,66}. Learn more at <https://docs.astral.sh/ruff> or in¹¹³.

Scikit-learn is a Python library offering various machine learning tools^{82,88}. Learn more at <https://scikit-learn.org>.

SciPy is a Python library for scientific computing^{47,110}. Learn more at <https://scipy.org>.

SGA The Standard Genetic Algorithm^{3,18,33,71,72,112} was the first population EA. It maintains a population of solutions and applies mutation and crossover to generate offspring solutions. It uses fitness proportionate selection to choose which solutions should „survive“ into the next generation, which today is considered a very bad design choice, see, e.g.,¹¹⁷.

SimPy is a Python library for discrete event simulation¹²². Learn more at <https://simpy.readthedocs.io>.

(string) interpolation In Python, string interpolation is the process where all the expressions in an f-string are evaluated and the final string is constructed. An example for string interpolation is turning `f"Rounded {1.234:.2f}"` to `"Rounded 1.23"`.

Glossary (in English) V



TensorFlow is a Python library for implementing machine learning, especially suitable for training of neural networks^{1,52}. Learn more at <https://www.tensorflow.org>.

TSP In an instance of the *Traveling Salesperson Problem*, also known as *Traveling Salesman Problem*, a set of n cities or locations as well as the distances between them are defined^{2,39,54,58,114,116}. The goal is to find the shortest round-trip tour that starts at one city, visits all the other cities one time each, and returns to the origin. The TSP is one of the most well-known $\mathcal{NP}$ -hard combinatorial optimization problems^{30,39}.

TTP The *Traveling Tournament Problem* (TTP) is the combinatorial optimization problem of both efficiently and fairly organizing a tournament of n teams that play against each other in a pairwise fashion^{27,119}. The efficient part boils down to arranging the games such that the total travel length is short, which is somewhat similar to the classical Traveling Salesperson Problem (TSP). Initially, each team is at its home location. On each day, a team needs to travel if its scheduled game is not at its present location. On the last day, each team may need to travel back home unless their last game is a home game. The total travel length sums up the lengths of all travels over all teams. The fair part is represented in several constraints, such as `doubleRoundRobin`, `compactness`, `maxStreak`, and `noRepeat`. The TTP is $\mathcal{NP}$ -hard¹⁰⁹.

unit test Software development is centered around creating the program code of an application, library, or otherwise useful system. A *unit test* is an *additional* code fragment that is not part of that productive code. It exists to execute (a part of) the productive code in a certain scenario (e.g., with specific parameters), to observe the behavior of that code, and to compare whether this behavior meets the specification^{6,74,76,78,92,103}. If not, the unit test fails. The use of unit tests is at least threefold: First, they help us to detect errors in the code. Second, program code is usually not developed only once and, from then on, used without change indefinitely. Instead, programs are often updated, improved, extended, and maintained over a long time. Unit tests can help us to detect whether such changes in the program code, maybe after years, violate the specification or, maybe, cause another, depending, module of the program to violate its specification. Third, they are part of the documentation or even specification of a program.

VCS A *Version Control System* is a software which allows you to manage and preserve the historical development of your program code¹⁰⁶. A distributed VCS allows multiple users to work on the same code and upload their changes to the server, which then preserves the change history. The most popular distributed VCS is Git.

Glossary (in English) VI



WWW World Wide Web^{7,20}

$\mathcal{NP}$ is the class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (such as a normal computer)³⁴.

$\mathcal{NP}$ -complete A decision problem is $\mathcal{NP}$ -complete if it is in $\mathcal{NP}$ and all problems in $\mathcal{NP}$ are reducible to it in polynomial time^{34,87}. A problem is $\mathcal{NP}$ -complete if it is $\mathcal{NP}$ -hard and if it is in $\mathcal{NP}$.

$\mathcal{NP}$ -hard Algorithms that guarantee to find the correct solutions of $\mathcal{NP}$ -hard problems^{14,17,53} have a worst-case runtime that grows exponentially with the problem scale. A problem is $\mathcal{NP}$ -hard if all problems in $\mathcal{NP}$ are reducible to it in polynomial time³⁴.